

# 目录

|                          |           |
|--------------------------|-----------|
| <b>第一章 二维数组1</b>         | <b>2</b>  |
| 1.1 二维数组 . . . . .       | 2         |
| 1.1.1 二维数组的本质 . . . . .  | 2         |
| 1.1.2 二维数组的初始化 . . . . . | 2         |
| 1.2 二维数组的遍历 . . . . .    | 4         |
| 1.3 二维数组的模拟 . . . . .    | 6         |
| <b>第二章 二维数组2</b>         | <b>9</b>  |
| 2.1 二维数组模拟 . . . . .     | 9         |
| 2.2 高维数组 . . . . .       | 15        |
| <b>第三章 转圈循环</b>          | <b>16</b> |
| <b>第四章 算法与复杂度</b>        | <b>21</b> |
| 4.1 算法与复杂度 . . . . .     | 21        |
| 4.1.1 什么是算法 . . . . .    | 21        |
| 4.1.2 算法的效率 . . . . .    | 22        |
| 4.1.3 复杂度的评估 . . . . .   | 23        |
| 4.1.4 大 O 表示法 . . . . .  | 23        |
| 4.1.5 复杂度的比较 . . . . .   | 23        |
| 4.2 模拟识别小数据 . . . . .    | 25        |
| <b>第五章 复杂模拟</b>          | <b>31</b> |
| <b>第六章 阶段练习1</b>         | <b>39</b> |
| <b>第七章 递归</b>            | <b>40</b> |
| 7.1 函数递归 . . . . .       | 40        |
| 7.2 搜索 . . . . .         | 44        |
| <b>第八章 step型搜索</b>       | <b>50</b> |
| 8.1 step型搜索 . . . . .    | 50        |

|                      |    |
|----------------------|----|
| 8.2 剪枝 . . . . .     | 53 |
| 第九章 加参数剪枝            | 57 |
| 第十章 回溯法              | 64 |
| 10.1 全排列枚举 . . . . . | 64 |
| 第十一章 回溯搜索练习          | 69 |
| 第十二章 阶段练习2           | 76 |

# 第一章 二维数组1

到现在为止，我们用的数组均为一维数组，为若干个同类型的变量组成的整体。但是，有些需要批量存储的问题似乎还是很难解决，例如储存一个表格里所有数时，我们更自然的说法是说这个数是第几行第几列，而非只说是第几个。这时，我们会使用二维数组。二维数组是多个同类型、同大小的一维数组组成的整体。在实际操作中，一维、二维数组最直观、最常用。

## 1.1 二维数组

将编号批量储存的思想推广到数组，我们可以将数组也进行编号并批量存储。与一维数组类似，C++ 中二维数组定义的格式为：

类型 数组名[下标上界1][下标上界2];

例如：

```
1 int array[1005][1005];
```

二维数组的所有其它定义规则均与一维数组相同，不再赘述。

### 1.1.1 二维数组的本质

我们定义一个  $n \times m$  的二维数组 `array[n][m]`，相当于定义了  $n$  个大小为  $m$  的一维数组 `array[0]`，`array[1]`，... `array[n-1]`。从这个角度，二维数组可以被理解为“数组的数组”。

二维数组还有一种更加直观的理解方式。一个  $n \times m$  的二维数组 `array[n][m]`，相当于一个  $n \times m$  的表格，而二维数组的元素 `array[i][j]` 则表示坐标为  $(i, j)$  的变量。事实上当我们需要查看二维数组的情况时，常常将二维数组输出成表格的形式；二维数组也常常用于存储图片等二维的数据。

### 1.1.2 二维数组的初始化

二维数组的初始化与一维数组类似，作为“数组的数组”，对二维数组赋初值时需要将每个“元素数组”初始化，例如：

```
1 int array[5][5]={{1,2,3}, {4,5,6}, {7,8,9}};
```

与一维数组类似，只要在初始化的时候赋值，那么所有未赋值的元素都会变成 0。

### 例题 1.1 植树大师

#### 【题目描述】

校门外整齐的种了一些树，这些树排列为  $n$  行  $m$  列，并且编号为  $1 \sim (n \times m)$ 。不过这些树并非根据编号排列的。

植树大师小然检查了一下数的编号，他不喜欢，于是决定把两棵树的位置互换一下。你现在知道树的初始排列方式，又知道小然打算把哪两个位置的树互换，你能预知换完之后树的排列方式吗？

#### 【输入格式】

输入由  $n + 2$  行组成：

第 1 行，2 个以空格隔开的正整数  $n, m$ ，表示数共有  $n$  行  $m$  列。

之后  $n$  行，依次表示第  $1, 2, \dots, n$  行的树；其中每行  $m$  个以空格隔开的正整数，表示这一行的第  $1, 2, \dots, m$  颗树的编号。保证编号是  $1 \sim (n \times m)$  各一个。

之后 1 行，4 个以空格隔开的正整数  $r1, c1, r2, c2$ ，表示小然打算把第  $r1$  行第  $c1$  列的树与第  $r2$  行第  $c2$  列的树交换。

#### 【输出格式】

输出展示了交换后树的排列情况：

输出共  $n$  行，依次表示第  $1, 2, \dots, n$  行的树；其中每行  $m$  个以空格隔开的正整数，表示这一行的第  $1, 2, \dots, m$

#### 【输入样例#1】

```
3 3
1 2 3
4 5 6
9 8 7
1 1 2 3
```

#### 【输出样例#1】

```
6 2 3
4 5 1
9 8 7
```

#### 【数据说明】

$3 \leq r1, r2 \leq n \leq 20$ ； $3 \leq c1, c2 \leq m \leq 20$ ；

所有编号是  $1 \sim (n \times m)$  范围内的正整数，且保证是  $1 \sim (n \times m)$  各一个。

## 1.2 二维数组的遍历

一维数组可以看成一条射线，而二维数组则可以看做一个二维的平面。沿着射线走只有一个方向，但在平面上走却有许多走法。对于不同的问题，我们常需要用不同的方式遍历二维数组，这一般需要通过循环来完成，其难点是找出循环变量和下标的关系。

### 例题 1.2 情人的视线

#### 【题目描述】

一幅画是由  $n$  行  $m$  列的 0,1 组成的矩形,画很美,而你却只关心画中的一个人。这个人出现的位置是第  $n1$  行到第  $n2$  行,第  $m1$  列到第  $m2$  列所组成的矩形。读入这幅画,并输出画中人。

#### 【输入格式】

输入由  $n + 1$  行组成

第一行为 6 个正整数  $n, m, n1, n2, m1, m2$

接下来的  $n$  行每行有  $m$  个正整数,代表画中每一行的数字

#### 【输出格式】

输出由  $n2 - n1 + 1$  行组成,每行有  $m2 - m1 + 1$  个数字组成,这是“画中人”所占据的矩形。

#### 【输入样例#1】

```
8 6 2 7 3 5
0 0 0 0 0 0
0 0 1 1 1 0
0 0 1 1 1 0
0 0 0 1 0 0
0 0 1 1 1 0
0 0 0 1 0 0
0 0 1 0 1 0
0 0 0 0 0 0
```

#### 【输出样例#1】

```
1 1 1
1 1 1
0 1 0
1 1 1
0 1 0
1 0 1
```

#### 【数据说明】

$5 \leq n, m \leq 30, 1 \leq n1 < n2 \leq n, 1 \leq m1 < m2 \leq m$ 。

## 例题 1.3 制作成绩单

## 【题目描述】

冷月老师每周都会给同学们布置作业。

冷月老师的班上有  $n$  名同学，本周冷月老师共布置了  $m$  道题目。给定每位同学每道题目的得分，冷月老师给每位同学发放成绩单，包含他自己的成绩和班级平均分。

## 【输入格式】

输入由  $n + 1$  行组成

第一行为两个正整数  $n, m$

接下来的  $n$  行每行有  $m$  个正整数，代表第  $n$  名同学每道题目的得分

## 【输出格式】

输入由  $n$  行组成

第  $i$  行包含第  $i$  个同学  $m$  道题每题得分及每题班级平均分（平均分四舍五入保留 2 位小数）

## 【输入样例#1】

```
3 4
80 90 90 50
100 100 100 100
60 60 100 30
```

## 【输出样例#1】

```
80(80.00) 90(83.33) 90(96.67) 50(60.00)
100(80.00) 100(83.33) 100(96.67) 100(60.00)
60(80.00) 60(83.33) 100(96.67) 30(60.00)
```

## 【数据说明】

$$1 \leq n \leq 30, 1 \leq m \leq 10$$

## 1.3 二维数组的模拟

有时候,我们需要使用二维数组来模拟一些操作,例如图像翻转,数据递推甚至模拟走迷宫等有趣的任务。这常需要对数组以特定的顺序进行大幅度的修改。

### 例题 1.4 情人的倒立

#### 【题目描述】

一幅画是由  $n$  行  $m$  列的 0,1 组成的矩形,画中人很美,所以你想知道她倒立起来是不是会别有一番风味,于是你决定把画进行上下翻转。

具体而言,对于  $n$  行的画,把第  $i$  行 ( $1 \leq i \leq n$ ) 和第  $n-i+1$  行对调就可以了!读入这幅画,输出画着倒立情人的画吧。

#### 【输入格式】

输入由  $n+1$  行组成:

第 1 行为 2 个正整数  $n, m$ , 表示画的尺寸为  $n$  行  $m$  列;

接下来的  $n$  行每行有  $m$  个用空格隔开的正整数,代表画中每一行的数字。

#### 【输出格式】

输出由  $n$  行组成:

每行有  $m$  个用空格隔开的数字,这是把画进行上下翻转后的样子。

#### 【输入样例#1】

```
6 6
0 0 1 1 1 0
0 0 1 1 1 0
0 0 0 1 0 0
0 0 1 1 1 0
0 0 0 1 0 0
0 0 1 0 1 0
```

#### 【输出样例#1】

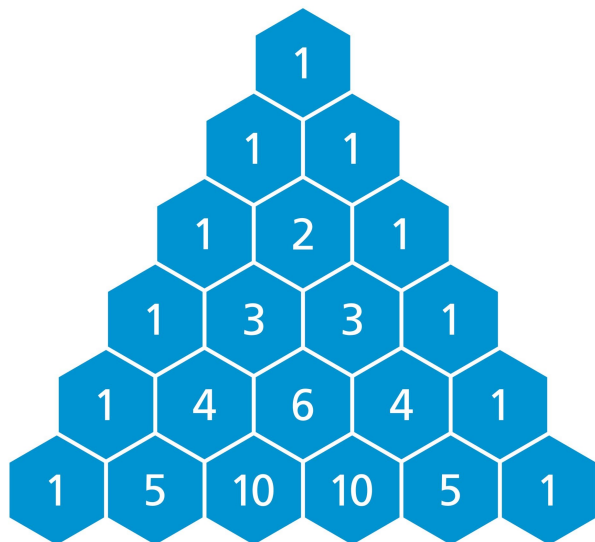
```
0 0 1 0 1 0
0 0 0 1 0 0
0 0 1 1 1 0
0 0 0 1 0 0
0 0 1 1 1 0
0 0 1 1 1 0
```

#### 【数据说明】

$5 \leq n, m \leq 30$ 。

### 例题 1.5 杨辉三角

杨辉三角是一类非常有趣的数阵。如图所示是一个 6 行的杨辉三角，杨辉三角最外层的数值均为 1，中间的数值则为正上方和左上角的两个数值相加。读入一个正整数  $n$ ，请编程打印一个  $n$  行的杨辉三角。



#### 【输入样例#1】

5

#### 【输出样例#1】

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```



## 例题 1.6 迷路的小猴

## 【题目描述】

一张方格表有  $n$  行  $m$  列的方格组成，而小猴站在左上角的格子中，他每次只能走到所在格子右边或下边相邻的格子中。但是有些格子中有坑，这些有坑格子是不能走的！

研究一下方格表的坑，然后告诉小猴现在他想走到右下角的格子有几种不同的走法吧！

## 【输入格式】

输入由  $n + 1$  行组成：

第 1 行为 2 个正整数  $n, m$ ，表示方格表的行数和列数。

第 2 ~  $n + 1$  行展示了方格表的"有坑"情况，每行  $m$  个用空格隔开的正整数 0 或 1，0 表示有坑，1 表示能走。

## 【输出格式】

1 个正整数，表示所求的走法数。

## 【输入样例#1】

```
3 4
1 0 1 1
1 1 1 1
1 1 1 1
```

## 【输出样例#1】

```
4
```

## 【数据说明】

$$2 \leq n, m \leq 10$$

## 第二章 二维数组2

### 2.1 二维数组模拟

例题 2.1 机器人犁田 The Robot Plow

#### 【题目描述】

Farmer John 为了让自己从无穷无尽的犁田工作中解放出来，于是买了个新机器人帮助他犁田。这个机器人可以完成犁田的任务，可惜有一个小小的缺点：这个犁田机器人一次只能犁一个边的长度是整数的长方形的田地。

因为 FJ 的田地有树和其它障碍物，所以 FJ 设定机器人去犁很多不同的长方形。这些长方形允许重叠。他给机器人下了  $I$  个指令，每个指令包含一个要犁长方形的地。这片田地由长方形的左下角和右上角坐标决定。他很好奇最后到底有多少个方格的地被犁过了。

一般来说，田地分割为很多小方格。这些方格的边和  $x$  轴或  $y$  轴平行。田地的宽度为  $X$  个方格，高度为  $Y$  个方格 ( $1 \leq X \leq 240$ ;  $1 \leq Y \leq 240$ )。FJ 执行了  $I$  ( $1 \leq I \leq 200$ ) 个指令，每个指令包含 4 个整数:  $Xll, Yll, Xur, Yur$  ( $1 \leq Xll \leq Xur \leq X$ ;  $1 \leq Yll \leq Yur \leq Y$ )，分别是要犁的长方形的左下角坐标和右上角坐标。机器人会犁所有的横坐标在  $Xll$  到  $Xur$  并且纵坐标在  $Yll$  到  $Yur$  范围内的所有方格的地。

考虑一个 6 方格宽 4 方格高的田地。FJ 进行了 2 个操作 (如下)，田地就被犁成 "\*" 和 "#" 了。虽然一般被犁过的地看起来都是一样的。但是标成 "#" 可以更清晰地看出最近一次被犁的长方形。

```
.....      **.....      #####.
..... (1,1)(2,4) **..... (1,3)(5,4) #####.
.....      **.....      **.....
.....      **.....      **.....
```

一共 14 个方格的地被犁过了。

#### 【输入格式】

第 1 行: 三个整数  $X, Y, I$

第 2 到  $I+1$  行: 第  $i+1$  行描述第  $i$  次犁田的指令，每行 4 个整数  $Xll, Yll, Xur, Yur$

#### 【输出格式】

一个整数，代表被犁过了格子数。

## 例题 2.2 铺地毯

## 【题目描述】

为了准备一个独特的颁奖典礼，组织者在会场的一片矩形区域（可看做是平面直角坐标系的第一象限）铺上一些矩形地毯。一共有  $n$  张地毯，编号从 1 到  $n$ 。现在将这些地毯按照编号从小到大的顺序平行于坐标轴先后铺设，后铺的地毯覆盖在前面已经铺好的地毯之上。

地毯铺设完成后，组织者想知道覆盖地面某个点的最上面的那张地毯的编号。注意：在矩形地毯边界和四个顶点上的点也算被地毯覆盖。

## 【输入格式】

输入共  $n + 2$  行

第一行，一个整数  $n$ ，表示总共有  $n$  张地毯

接下来的  $n$  行中，第  $i + 1$  行表示编号  $i$  的地毯的信息，包含四个正整数  $a, b, g, k$ ，每两个整数之间用一个空格隔开，分别表示铺设地毯的左下角的坐标  $(a, b)$  以及地毯在  $x$  轴和  $y$  轴方向的长度

第  $n + 2$  行包含两个正整数  $x$  和  $y$ ，表示所求的地面的点的坐标  $(x, y)$

## 【输出格式】

输出共 1 行，一个整数，表示所求的地毯的编号；若此处没有被地毯覆盖则输出 -1

## 【输入样例#1】

```
3
1 0 2 3
0 2 3 3
2 1 3 3
2 2
```

## 【输出样例#1】

```
3
```

## 【数据说明】

对于 50% 的数据， $0 \leq a, b, g, k \leq 100$ ；

对于 100% 的数据，有  $0 \leq n \leq 10,000$ ， $0 \leq a, b, g, k \leq 100,000$ 。

## 例题 2.3 轰炸II

## 【题目描述】

一个城市遭到了  $M$  次轰炸，每次都炸了一个每条边都与边界平行的矩形

在轰炸后，有  $N$  个关键点，指挥官想知道，它们有没有受到过轰炸，如果有，被炸了几次，最后一次是第几轮。

## 【输入格式】

第一行，两个整数： $M, N$

以下  $M$  行，每行四个整数： $x1, y1, x2, y2$ ，表示被轰炸的矩形的左上角坐标和右下角坐标（比如 1 3 7 10 就表示被轰炸的地方是从 (1, 3) 到 (7, 10) 的矩形）。

再以下  $N$  行，每行两个整数，表示这个关键点的坐标。

## 【输出格式】

共  $N$  行，

每行第一个字符串为 YES 或 NO，表示是否被轰炸，若为 YES，在一个空格后为两个整数，表示被炸了几次和最后一次是第几轮。

## 【输入样例#1】

```
1 1
1 1 10 10
11 11
```

## 【输出样例#1】

```
NO
```

## 【数据说明】

$$1 \leq N, M \leq 2000$$

$$1 \leq x1, y1, x2, y2 \leq 2147483647$$

## 例题 2.4 200704 温馨的家

## 【题目描述】

企鹅皮皮从小就是一个淘气的孩子。现在，他已经长大了，需要自己建一个新家了。

你知道企鹅的家是用小石块在地面上铺出来的吧？为了让自己的家看起来更漂亮，皮皮找了很多种不同颜色的石块，并把它们铺成一圈一圈的：先找某种颜色的一颗放中心，在它周围铺上一圈第二种颜色的，再在周围铺上一圈第三种颜色的……

这当然是一个漂亮的家。

你能用你的知识，让计算机画出皮皮的家么？

## 【输入格式】

只有一个整数  $n$  ( $1 \leq n \leq 26$ )，表示皮皮的家是由多少圈石块组成的

## 【输出格式】

输出皮皮家的示意图。用字母 "A" 表示第一种颜色的石块，用字母 "B" 表示第二种颜色的石块……以此类推

## 【输入样例#1】

4

## 【输出样例#1】

```
DDDDDDD
DCCCCCD
DCBBBCD
DCBABCD
DCBBBCD
DCCCCCD
DDDDDDD
```

## 例题 2.5 验证幻方

**【题目描述】**

幻方是另一类有趣的数阵，它是由若干个整数组成的正方形。对于一个幻方，它每一行、每一列和两条对角线上的数相加之和均为同一个值。给定一个正方形的数阵，请编程判断它是否是一个幻方。

**【输入格式】**

输入由  $n + 1$  行组成

第一行为一个正整数  $n$

接下来的  $n$  行为一个  $n \times n$  的正方形数阵，数据保证此数阵由 1 到  $n^2$  的所有整数组成

**【输出格式】**

若此数阵是一个幻方，输出 "Yes"；否则输出 "No"

## 例题 2.6 神奇的幻方

## 【题目描述】

幻方是一种很神奇的  $N \times N$  矩阵：它由数字  $1, 2, 3, \dots, N \times N$  构成，且每行、每列及两条对角线上的数字之和都相同。当  $N$  为奇数时，我们可以通过下方法构建一个幻方：

首先将 1 写在第一行的中间。

之后，按如下方式从小到大依次填写每个数  $K(K = 2, 3, \dots, N \times N)$ ：

1. 若  $(K - 1)$  在第一行但不在最后一列，则将  $K$  填在最后一行， $(K - 1)$  所在列的右一列；
2. 若  $(K - 1)$  在最后一列但不在第一行，则将  $K$  填在第一列， $(K - 1)$  所在行的上一行；
3. 若  $(K - 1)$  在第一行最后一列，则将  $K$  填在  $(K - 1)$  的正下方；
4. 若  $(K - 1)$  既不在第一行，也不在最后一列，如果  $(K - 1)$  的右上方还未填数，则将  $K$  填在  $(K - 1)$  的右上方，否则将  $K$  填在  $(K - 1)$  的正下方。

现给定  $N$ ，请按上述方法构造  $N \times N$  的幻方。

## 【输入格式】

输入只有一行，包含一个整数  $N$ ，即幻方的大小。

## 【输出格式】

共  $N$  行，每行  $N$  个整数，即按上述方法构造出的  $N \times N$  的幻方，相邻两个整数之间用单空格隔开。

## 【输入样例#1】

3

## 【输出样例#1】

8 1 6

3 5 7

4 9 2

## 【数据说明】

对于 100% 的数据， $1 \leq N \leq 39$  且  $N$  为奇数。

## 2.2 高维数组

依照习惯，三维及以上的数组被称为高维数组。多个同类型、同大小的  $N$  维数组所组成的集合就是  $N + 1$  维数组。实际操作中高维数组的应用不多，但有些时候运用高维数组可以巧妙地解决一些问题。

在 C++ 中，由于多维数组被定义为“数组的数组”，所以对于数组的维度并无限制，可以根据需要自行定义任意维度的数组。

### 例题 2.7 整队

#### 【题目描述】

军训中， $n$  个排各自组成了一个  $m \times k$  的方阵，并依次入场。现在，连长宣布每个方阵的第  $i$  行，第  $j$  列的人为方阵长，并让他们按方阵入场顺序在主席台前成一列，听候调遣。以士兵编号的形式给出每个方阵的人员，输出方阵长所组成的队列。

#### 【输入格式】

输入的第一行上有 3 个整数，依次为  $n, m, k$ 。之后  $n \times m$  行，每行  $k$  个以空格隔开的正整数，为按顺序入场的  $n$  个方阵的士兵编号。最后还有一行，为两个整数  $i, j$ 。

#### 【输出格式】

一行， $n$  个整数，各位方阵长的编号

#### 【输入样例#1】

```
3 4 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
41 42 43 44
45 46 47 48
49 50 51 52
53 54 55 56
101 102 103 104
105 106 107 108
109 110 111 112
113 114 115 116
2 3
```

#### 【输出样例#1】

```
7 47 107
```

#### 【数据说明】

$n, m, k \leq 20, i \leq m, j \leq k$ 。



## 第三章 转圈循环

### 例题 3.1 约瑟夫游戏

#### 【题目描述】

$n$  个人围成一圈，从第一个人开始报数,从 1 开始,数到  $m$  的人出列，再由下一个人重新从 1 开始报数，数到  $m$  的人再出列，依次类推，直到所有的人都出列，请输出依次出列人的编号。

#### 【输入格式】

输入两个整数  $n, m$ 。

#### 【输出格式】

输出一行  $n$  个整数，按出列顺序输出每个出列人的编号。

#### 【输入样例#1】

10 3

#### 【输出样例#1】

3 6 9 2 7 1 8 5 10 4

#### 【数据说明】

$1 \leq m, n \leq 100$

### 例题 3.2 圆环上的生命游戏

#### 【题目描述】

也许你听说过康威生命游戏 (Conway's Game of Life)。康威生命游戏适用于方格组成的矩阵。但它可以产生十分复杂的结构。在这道题目中, 我们将探讨简化版的生命游戏。

将一个圆环分为  $N$  段, 将这  $N$  段顺时针依次编为  $1, \dots, N$  号。每一段内有一个细胞, 要么是生存状态 (以 1 表示), 要么是死亡状态 (以 0 表示)。我们把每个细胞两边的细胞称作它的邻居。

在时刻 0, 给出那些活着的细胞的位置, 今后任何时刻的状态, 都被它前面的状态按照下面的游戏规则无情地规定下来了:

**出生:** 在时刻  $t$  处于死亡状态的细胞, 如果他的两个邻居中恰好有 1 个在时刻  $t$  活着, 那么这个细胞将在  $t+1$  时刻变成生存状态。

**死亡:** 在时刻  $t$  处于生存状态的细胞, 如果他的两个邻居中恰好有 0 个 (孤立) 或 2 个 (过于拥挤) 在时刻  $t$  活着, 那么这个细胞将在  $t+1$  时刻变成死亡状态。

**生存:** 在时刻  $t$  处于生存状态的细胞, 如果他的两个邻居中恰好有 1 个在时刻  $t$  活着, 那么这个细胞在  $t+1$  时刻仍然会继续保持生存状态。

给出  $N$  和时刻 0 时每个细胞的生存状态, 求出时刻  $T$  时细胞的生存状态

#### 【输入格式】

输入共两行:

第 1 行: 2 个正整数  $N, T$ , 以空格分隔。

第 2 行:  $N$  个正整数  $a_1, a_2, \dots, a_N$ , 以空格分隔,  $a_i$  表示细胞  $i$  在时刻 0 的状态。

$a_i = 0$  表示细胞  $i$  处于死亡状态,  $a_i = 1$  表示细胞  $i$  处于生存状态。

#### 【输出格式】

输出 1 行,  $N$  个数, 以空格分隔, 第  $i$  个数表示细胞  $i$  在时刻  $T$  的生存状态。(以 1 表示生存, 以 0 表示死亡)

#### 【输入样例 #1】

```
7 1
0 0 0 0 0 0 1
```

#### 【输出样例 #1】

```
1 0 0 0 0 1 0
```

#### 【数据说明】

$1 \leq N \leq 10000$

$1 \leq T \leq 1000$

**例题 3.3 循环数****【题目描述】**

循环数是那些不包括 0 且没有重复数字的整数（比如 81362）并且还应同时具有一个有趣的性质，就像这个例子：

如果你从最左边的数字开始（在这个例子中是 8）向右数最左边这个数（如果数到了最右边就回到最左边），你会停止在另一个新的数字（如果停在一个相同的数字上，这个数就不是循环数）。

就像：8 1 3 6 2 从最左边接下去数 8 个数字：1 3 6 2 8 1 3 6 所以下一个数字是 6

重复这样做（这次从 6 开始数 6 个数字）并且你会停止在一个新的数字上：2 8 1 3 6 2，也就是 2

再这样做（这次数两个）：8 1

再一次（这次一个）：3

又一次：6 2 8

这时你回到了起点，在经过每个数字一次后回到起点的就是循环数。如果你经过每一个数字一次以后没有回到起点，你的数字不是一个循环数。

给你一个数字  $m$ ，找出第一个比  $m$  大的循环数，输出数据保证结果能用一个无符号长整型数  $\in [0, 2^{32})$  装下。（追加提醒：循环数每个数位都必须访问到）

**【输入格式】**

仅仅一行，包括  $m$ 。

**【输出格式】**

仅仅一行，输出第一个比  $m$  大的循环数。

**【输入样例#1】**

81361

**【输出样例#1】**

81362

**【数据说明】**

对于 100% 的数据， $1 \leq m \leq 10^9$ 。

### 例题 3.4 寻宝

#### 【题目描述】

传说很遥远的藏宝楼顶层藏着诱人的宝藏。小明历尽千辛万苦终于找到传说中的这个藏宝楼，藏宝楼的门口竖着一个木板，上面写有几个大字：寻宝说明书。

说明书的内容如下：

藏宝楼共有  $N+1$  层，最上面一层是顶层，顶层有一个房间里面藏着宝藏。除了顶层外，藏宝楼另有  $N$  层，每层  $M$  个房间，这  $M$  个房间围成一圈并按逆时针方向依次编号为  $0, \dots, M-1$ 。其中一些房间有通往上一层的楼梯，每层楼的楼梯设计可能不同。

每个房间里有一个指示牌，指示牌上有一个数字  $x$ ，表示从这个房间开始按逆时针方向选择第  $x$  个有楼梯的房间（假定被选择的房间的编号为  $k$ ），从该房间上楼，上楼后到达上一层的  $k$  号房间。

比如当前房间的指示牌上写着 2，则按逆时针方向开始尝试，找到第 2 个有楼梯的房间，从该房间上楼。如果当前房间本身就有楼梯通向上层，该房间作为第 1 个有楼梯的房间。

寻宝说明书的最后用红色大号字体写着：“寻宝须知：帮助你找到每层上楼房间的指示牌上的数字（即每层第一个进入的房间内指示牌上的数字）总和为打开宝箱的密钥”。

请帮助小明算出这个打开宝箱的密钥。

#### 【输入格式】

第一行 2 个整数  $N$  和  $M$ ，之间用一个空格隔开。 $N$  表示除了顶层外藏宝楼共  $N$  层楼， $M$  表示除顶层外每层楼有  $M$  个房间。

接下来  $N \times M$  行，每行两个整数，之间用一个空格隔开，每行描述一个房间内的情况，其中第  $(i-1)M+j$  行表示第  $i$  层  $j-1$  号房间的情况（ $i=1, 2, \dots, N; j=1, 2, \dots, M$ ）。

每行第一个整数表示该房间是否有楼梯通往上一层（0 表示没有，1 表示有），第二个整数表示指示牌上的数字。注意，从  $j$  号房间的楼梯爬到上一层到达的房间一定也是  $j$  号房间。

最后一行，一个整数，表示小明从藏宝楼底层的几号房间进入开始寻宝（注：房间编号从 0 开始）。

#### 【输出格式】

输出只有一行，一个整数，表示打开宝箱的密钥，这个数可能会很大，请输出对 20123 取模的结果即可。

#### 【输入样例 #1】

```
2 3
1 2
0 3
1 4
0 1
1 5
1 2
1
```

**【输出样例#1】**

5

**【数据说明】**

对于 50% 数据，有  $0 < N \leq 1000$ ， $0 < x \leq 10000$ ；

对于 100% 数据，有  $0 < N \leq 10000$ ， $0 < M \leq 100$ ， $0 < x \leq 1,000,000$ 。

## 第四章 算法与复杂度

### 4.1 算法与复杂度

掌握编程语言的基本语法规则后，便可以像搭积木一样对其进行拼接组合，从而解决各种不同类型的问题。在解决问题时，不仅需要特定语言的语法知识，更需要搞清“问题该按怎样的步骤解决”——即具备设计算法的能力。

#### 4.1.1 什么是算法

算法由英文 Algorithm 直译而来。即“运算法则”。广义上讲，就是指“达成某种目的的步骤”。严格来说，算法是解题方案的准确而完整的描述，是一系列解决问题的清晰指令，算法代表着用系统的方法描述解决问题的策略机制。也就是说，算法就是解决问题的办法，能够对一定规范的输入，在有限时间内获得所要求的输出。常见的算法有递推法、递归法、穷举法、贪心算法、分治法、动态规划法、回溯法等。

算法具有三个特性：

- (1) 有穷性：一个算法必须能够对任何合法的输入在执行有穷步之后结束，且每一步都可在有穷时间内完成；
- (2) 确切性：算法中每一条指令必须有确切的含义，读者理解时不会产生歧义。并且，在任何条件下，算法只有唯一的一条执行路径，即对于相同的输入只能得出相同的输出；
- (3) 可行性：算法中描述的操作都是可以通过已经实现的基本操作执行有限次来实现。

一般来说，一个问题都对应着多种算法，不同的算法所需的运行时间也天差地别。开始新系统，或者利用、借鉴前人们研究开发的各种古典算法，都可以高效地解决问题。当我们需要自己设计算法时，也可以借鉴现有算法的思路。

**例题 4.1** 读取  $n$  名选手的得分数据，按照由高到低的顺序输出前  $m$  名的得分。注：满分为  $t$  分，且得分都为正整数。

现在要考虑如何用程序来实现这一问题。不难发现，即使是这么简单的一个题目，我们也能列举出多种算法。

## 算法一：m 次搜索

1. 将各选手的得分输入数组  $A_1, A_2, \dots, A_N$
2. 找出  $A$  中的最大值并输出
3. 在  $A$  中剔除 2 中选定的元素
4. 重复 2、3 步，直到一共输出  $m$  个数

## 算法二：排序后输出前 3 名得分

1. 将各选手的得分输入数组  $A_1, A_2, \dots, A_N$
2. 将  $A$  按降序排列
3. 按顺序输出  $A$  中的前  $m$  个元素

## 算法三：统计各个得分的人数

1. 将得分为  $p$  的人数计入数组  $\text{cnt}[p]$
2. 按  $\text{cnt}[100], \text{cnt}[99] \dots$  的顺序，当  $\text{cnt}[p]$  大于等于 1 时，将  $p$  输出  $\text{cnt}[p]$  次，输出  $m$  次后停止

算法 1 是最直接，也最容易想到的，在  $m$  很小时效率高。算法 2 如果可以借助现成的排序算法，那么实现算法 2 的代码量极短，效率也很高。算法 3 在  $t$  较小时算法效率很高，但在  $t$  很大的情况下会占去非常多的存储空间，反而不实用。

可见，程序设计问题的解题方法并不唯一，经常有多个算法同时适用于一道题目。所以，我们在考虑采用哪种算法时要先考虑数据的范围。程序简洁程度、编码难易程度、运算效率、内存使用量都是选择算法的标准，但后两个要远比前两个重要。

### 4.1.2 算法的效率

在我们看来，计算机只需一眨眼的功夫便能完成运算处理。但实际上，它也需要根据输入数据的大小和算法效率消耗一定量的处理器资源。要想编写出一段能高速运行的优秀程序，我们必须充分考量问题的性质，力求让所选算法能高效处理一切有可能输入的数据。

考虑解题的算法时，我们需要一个“标尺”来衡量其效率。在设计算法并编写程序之前，我们应该先推测其效率，看一看它是不是足以让我们解决眼前问题。如果忽视这一过程，很有可能辛辛苦苦编写出来的程序并不符合要求，导致白费功夫。



当今计算机处理能力的发展速度可谓一日千里，但与此同时，需要处理的数据量和复杂度也与日俱增。在这个要求活用大数据、拓展计算机用途、提高算法效率以求节能降耗等的大背景下，开发高效算法的研究项目正遍地开花。

### 4.1.3 复杂度的评估

算法的效率主要由以下两个复杂度来评估：

- 时间复杂度：评估执行程序所需时间，可以估算出程序对计算机处理器的使用程度。
- 空间复杂度：评估执行程序所需的存储空间，可以估算出程序对计算机内存的使用程度。

由于近年来硬件发展迅速，时间复杂度往往比空间复杂度更容易超出限制，因此不特别说明的情况下，说“算法复杂度”都是指“时间复杂度”。

### 4.1.4 大 O 表示法

评估时间复杂度时，需要统计程序中“基本操作”的数量，这样可以排除掉机器速度的影响，只衡量算法本身的优劣程度。例如四则运算、比较运算都属于基本操作。

大 O 表示法是一种评估算法效率的“标尺”。在该表示法中，我们以诸如  $O(n)$ 、 $O(n^2)$  的形式来表示算法的效率，其中  $n$  为问题的输入数据规模。 $O(g(n))$  代表该算法复杂度与  $g(n)$  同阶。

同阶是什么意思呢？简单地说，就是“增长情况相同”。例如程序基本操作数是  $3n^3 - 100n^2 + 5n$ ，当  $n$  很大时，只有立方项起到决定作用，而立方项的系数对“增长”是不起作用的—— $n$  扩大两倍时， $n^3$  和  $3n^3$  都扩大 8 倍。这样一来，可以只保留“最大项”，并忽略其系数，得到的简单式子称为算法的渐进时间复杂度（asymptotic time complexity），也就是大 O 表示法。

算法的复杂度可以从最理想、平均情况和最坏情况三种角度进行估算。由于平均情况的复杂度大多和最坏情况的持平，而且估算最坏情况的复杂度可以避免后顾之忧，因此一般情况下，我们在设计算法时都直接估算最坏情况的复杂度。

### 4.1.5 复杂度的比较

以现代计算机的处理能力来看，1s 的时间最多支持  $10^8$  量级的简单运算，信息竞赛中对内存限制一般支持数组开到  $10^7$  量级。

| 数据范围            | 算法最大量级        | 常见算法类型        |
|-----------------|---------------|---------------|
| $n < 20$        | $O(2^n)$      | 搜索,状压DP       |
| $n < 500$       | $O(n^3)$      | Floyd,三维      |
| $n < 10000$     | $O(n^2)$      | 图论,DP         |
| $n < 300000$    | $O(n \log n)$ | 二分,倍增,排序,数据结构 |
| $n < 100000000$ | $O(n)$        | 数学,贪心         |



## 例题 4.2 秦九韶算法

计算多项式的值：

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

其中  $1 \leq n \leq 10^5$ 。比较以下算法的复杂度。

## 算法一：依次计算

对  $i = 0, 1, \cdots, n$  分别计算  $a_i x^i$  的值，需要  $i$  次乘法。计算完所有项需要的乘法次数是  $O(n^2)$  的。

## 算法二：秦九韶算法

作如下转化：

$$f(x) = (a_n x^{n-1} + a_{n-1} x^{n-2} + \cdots + a_1)x + a_0$$

把常数项  $a_0$  先扔在一边，其他项都可以提取出一个公因式  $x$ ，括号内是一个  $n-1$  次多项式。于是可以先计算出括号内的值，再乘  $x$  再加  $a_0$ 。

计算括号内的  $n-1$  次多项式时，也如法炮制，直到括号内变成  $a_n x + a_{n-1}$ 。

实际算法流程和推导过程正好相反：

1. 先令  $f = 0$ 。
2. 对  $i = n \sim 0$ ：令  $f$  等于  $f \times x + a_i$ 。
3.  $f$  最终的值为答案。

## 4.2 模拟识别小数据

现在我们学会了如何分析算法复杂度。在具体问题中，程序运行时间和数据规模有关，从数据规模较小的变量入手，更容易设计出高效的算法。

### 例题 4.3 公交换乘

#### 【题目描述】

著名旅游城市 B 市为了鼓励大家采用公共交通方式出行，推出了一种地铁换乘公交车的优惠方案：

1. 在搭乘一次地铁后可以获得一张优惠票，有效期为 45 分钟，在有效期内可以消耗这张优惠票，免费搭乘一次票价不超过地铁票价的公交车。在有效期内指开始乘公交车的时间与开始乘地铁的时间之差小于等于 45 分钟，即： $t_{bus} - t_{subway} \leq 45$

2. 搭乘地铁获得的优惠票可以累积，即可以连续搭乘若干次地铁后再连续使用优惠票搭乘公交车。

3. 搭乘公交车时，如果可以使用优惠票一定会使用优惠票；如果有多张优惠票满足条件，则优先消耗获得最早的优惠票。

现在你得到了小轩最近的公共交通出行记录，你能帮他算算他的花费吗？

#### 【输入格式】

输入的第一行包含一个正整数  $n$ ，代表乘车记录的数量。

接下来的  $n$  行，每行包含 3 个整数，相邻两数之间以一个空格分隔。第  $i$  行的第 1 个整数代表第  $i$  条记录乘坐的交通工具，0 代表地铁，1 代表公交车；第 2 个整数代表第  $i$  条记录乘车的票价  $price_i$ ；第三个整数代表第  $i$  条记录开始乘车的时间  $t_i$ （距 0 时刻的分钟数）。

我们保证出行记录是按照开始乘车的时间顺序给出的，且不会有两次乘车记录出现在同一分钟。

#### 【输出格式】

输出有一行，包含一个正整数，代表小轩出行的总花费。

#### 【输入样例#1】

```
6
0 10 3
1 5 46
0 12 50
1 3 96
0 5 110
1 6 135
```

#### 【输出样例#1】

```
36
```

#### 【输入样例#2】

6  
0 5 1  
0 20 16  
0 7 23  
1 18 31  
1 4 38  
1 7 68

### 【输出样例#2】

32

### 【样例说明】

样例#1:

第一条记录, 在第 3 分钟花费 10 元乘坐地铁。

第二条记录, 在第 46 分钟乘坐公交车, 可以使用第一条记录中乘坐地铁获得的优惠票, 因此没有花费。

第三条记录, 在第 50 分钟花费 12 元乘坐地铁。

第四条记录, 在第 96 分钟乘坐公交车, 由于距离第三条记录中乘坐地铁已超过 45 分钟, 所以优惠票已失效, 花费 3 元乘坐公交车。

第五条记录, 在第 110 分钟花费 5 元乘坐地铁。

第六条记录, 在第 135 分钟乘坐公交车, 由于此时手中只有第五条记录中乘坐地铁获得的优惠票有效, 而本次公交车的票价为 6 元, 高于第五条记录中地铁的票价 5 元, 所以不能使用优惠票, 花费 6 元乘坐公交车。

总共花费 36 元。

样例#2:

第一条记录, 在第 1 分钟花费 5 元乘坐地铁。

第二条记录, 在第 16 分钟花费 20 元乘坐地铁。

第三条记录, 在第 23 分钟花费 7 元乘坐地铁。

第四条记录, 在第 31 分钟乘坐公交车, 此时只有第二条记录中乘坐的地铁票价高于本次公交车票价, 所以使用第二条记录中乘坐地铁获得的优惠票。

第五条记录, 在第 38 分钟乘坐公交车, 此时第一条和第三条记录中乘坐地铁获得的优惠票都可以使用, 使用获得最早的优惠票, 即第一条记录中乘坐地铁获得的优惠票。

第六条记录, 在第 68 分钟乘坐公交车, 使用第三条记录中乘坐地铁获得的优惠票。

总共花费 32 元。

### 【数据说明】

对于 30% 的数据,  $n \leq 1000$ ,  $t_i \leq 10^6$ 。

另有 15% 的数据,  $t_i \leq 10^7$ ,  $price_i$  都相等。

另有 15% 的数据,  $t_i \leq 10^9$ ,  $price_i$  都相等。

对于 100% 的数据,  $n \leq 10^5$ ,  $t_i \leq 10^9$ ,  $1 \leq price_i \leq 1000$ 。

#### 例题 4.4 牌力计算

##### 【题目描述】

小明在玩一个很奇怪的卡牌游戏，这个游戏的每张牌的点数都是一个正整数，小明手里有  $N$  张牌的时候，手牌的牌力按如下规则计算：

牌力初始为 0，将所有牌按照点数从小到大排序后，对于每对相邻的牌：如果这两张相邻的牌点数相等，牌力增加  $k$ ；如果这两张牌点数不相等，那么牌力增加两张牌点数差的平方。

例如 5 张牌从小到大为 1, 1, 3, 3, 4，且  $k = 3$ ，那么牌力为  $3 + (1 - 3)^2 + 3 + (4 - 3)^2 = 11$  点。

但是  $N$  张牌是一张张依次发给小明的，小明想要知道当他手中有  $1, 2, \dots, N$  张牌时，牌力分别是多少？（只有 1 张牌时的牌力认为是 0）

##### 【输入格式】

第 1 行，2 个正整数  $N, k$ ，用空格分隔（ $1 \leq N \leq 10^5$ ； $1 \leq k \leq 10^9$ ）

接下来  $N$  行，每行一个正整数，第  $i + 1$  行的正整数  $a_i$  表示第  $i$  张发给小明的牌的点数。（ $1 \leq a_i \leq 500$ ）

##### 【输出格式】

输出  $N$  行，第  $i$  行表示小明拿到前  $i$  张牌时，他手牌的牌力。

##### 【输入样例#1】

```
5 3
4
1
3
3
1
```

##### 【输出样例#1】

```
0
9
5
8
11
```

##### 【样例说明】

小明会先后拿到点数为 1, 1, 3, 3, 4 的牌。 $k = 3$

拿到第 1 张牌后，小明的手牌为 4，牌力为 0

拿到第 2 张牌后，小明的手牌为 1, 4，牌力为  $(1 - 4)^2 = 9$

拿到第 3 张牌后，小明的手牌为 1, 3, 4，牌力为  $(1 - 3)^2 + (3 - 4)^2 = 5$

拿到第 4 张牌后，小明的手牌为 1, 3, 3, 4，牌力为  $(1 - 3)^2 + 3 + (3 - 4)^2 = 8$

拿到第 5 张牌后，小明的手牌为 1, 1, 3, 3, 4，牌力为  $3 + (1 - 3)^2 + 3 + (4 - 3)^2 = 11$

## 例题 4.5 超级斗地主

## 【题目描述】

小明在玩超级斗地主，超级斗地主的牌的点数可能是从 1 到 5000 的任意一个数。小明的手牌有  $N$  张。他想通过出一个“ $k$  连顺”的方式，一次打出大量的手牌。“ $k$  连顺”定义如下：

一个“ $k$  连顺”包含  $k$  种不同的点数，这些点数必须是连续的，并且每种点数的牌的数量必须相同。

例如：333444555 就是一个“3 连顺”，包含 9 张牌；56789 是一个“5 连顺”，包含 5 张牌；

999999999999 是一个“1 连顺”，包含 12 张牌。

给出小明的所有手牌，他手中包含卡牌数量最多的“ $k$  连顺”包含多少张牌？

## 【输入格式】

第 1 行，1 个正整数  $N$ ，表示小明的手牌张数

第 2 行， $N$  个正整数，用空格分隔，表示小明所有手牌的点数

## 【输出格式】

输出一个正整数，卡牌数量最多的“ $k$  连顺”包含的张数

## 【输入样例#1】

17

3 3 4 4 5 5 6 7 8 9 9 9 9 9 9 9 9

## 【输出样例#1】

8

## 【样例说明】

可能的“ $k$  连顺”有：334455，3456789，99999999。其中 99999999 包含了 8 张牌，是包含牌数量最多的“ $k$  连顺”。

## 【数据说明】

对于 20% 的数据，所有牌点数互不相同。

对于 60% 的数据， $1 \leq N \leq 10^3$ 。

对于 100% 的数据， $1 \leq N \leq 10^6$ ，小明的手牌的点数不超过 5000

## 例题 4.6 直播获奖

### 【题目描述】

NOI2130 即将举行。为了增加观赏性，CCF 决定逐一评出每个选手的成绩，并直播即时的获奖分数线。本次竞赛的获奖率为  $w\%$ ，即当前排名前  $w\%$  的选手的最低成绩就是即时的分数线。

更具体地，若当前已评出了  $p$  个选手的成绩，则当前计划获奖人数为  $\max(1, \lfloor p * w\% \rfloor)$ ，其中  $w$  是获奖百分比， $\lfloor x \rfloor$  表示对  $x$  向下取整， $\max(x, y)$  表示  $x$  和  $y$  中较大的数。如有选手成绩相同，则所有成绩并列的选手都能获奖，因此实际获奖人数可能比计划中多。

作为评测组的技术人员，请你帮 CCF 写一个直播程序。

### 【输入格式】

第一行有两个整数  $n, w$ 。分别代表选手总数与获奖率。

第二行有  $n$  个整数，依次代表逐一评出的选手成绩。

### 【输出格式】

只有一行，包含  $n$  个非负整数，依次代表选手成绩逐一评出后，即时的获奖分数线。相邻两个整数间用一个空格分隔。<

### 【输入样例#1】

```
10 60
200 300 400 500 600 600 0 300 200 100
```

### 【输出样例#1】

```
200 300 400 400 400 500 400 400 300 300
```

### 【输入样例#2】

```
10 30
100 100 600 100 100 100 100 100 100 100
```

### 【输出样例#2】

```
100 100 600 600 600 600 100 100 100 100
```

### 【样例说明】

|                                           |            |            |            |            |            |            |            |            |            |            |
|-------------------------------------------|------------|------------|------------|------------|------------|------------|------------|------------|------------|------------|
| 已评测选手人数                                   | 1          | 2          | 3          | 4          | 5          | 6          | 7          | 8          | 9          | 10         |
| 计划获奖人数                                    | 1          | 1          | 1          | 2          | 3          | 3          | 4          | 4          | 5          | 6          |
| 已评测选手的分数从高到低排列<br>(其中, 分数线用 <b>粗体</b> 标出) | <b>200</b> | <b>300</b> | <b>400</b> | 500        | 600        | 600        | 600        | 600        | 600        | 600        |
|                                           |            | 200        | 300        | <b>400</b> | 500        | 600        | 600        | 600        | 600        | 600        |
|                                           |            |            | 200        | 300        | <b>400</b> | <b>500</b> | 500        | 500        | 500        | 500        |
|                                           |            |            |            | 200        | 300        | 400        | <b>400</b> | <b>400</b> | 400        | 400        |
|                                           |            |            |            |            | 200        | 300        | 300        | 300        | <b>300</b> | 300        |
|                                           |            |            |            |            |            | 200        | 200        | 300        | 300        | <b>300</b> |
|                                           |            |            |            |            |            |            | 0          | 200        | 200        | 200        |
|                                           |            |            |            |            |            |            |            | 0          | 200        | 200        |
|                                           |            |            |            |            |            |            |            |            | 0          | 100        |
|                                           |            |            |            |            |            |            |            |            |            | 0          |

## 【数据说明】

|         |          |
|---------|----------|
| 测试点编号   | $n$      |
| 1 ~ 3   | = 10     |
| 4 ~ 6   | = 500    |
| 7 ~ 10  | = 2000   |
| 11 ~ 17 | = 10000  |
| 18 ~ 20 | = 100000 |

对于所有测试点, 每个选手的成绩均为不超过 600 的非负整数, 获奖百分比  $w$  是一个正整数且  $1 \leq w \leq 99$ 。

## 第五章 复杂模拟

例题 5.1 ABC045B 三人卡片游戏.

### 【题目描述】

A,B,C 三人在玩一个卡片游戏, 规则如下:

- 游戏开始时, 3 人分别有卡片若干枚, 每张卡上写着 ‘a’, ‘b’, ‘c’ 中某一个字母。卡片的顺序严格按照输入顺序排列, 不允许改变顺序。
- 游戏从 A 的回合开始。
- 如果轮到某人的回合, 且这个人手上有牌, 他必须出自己手上的第 1 张牌。之后轮到这张牌的写着的字母同名的人的回合。(例如出 ‘a’ 牌之后就轮到 A 的回合)
- 如果轮到某人的回合, 且这个人手没有牌, 这个人就是赢家。

三人的起始手牌以字符串  $S_A, S_B, S_C$  的形式给出, 字符串开头是第 1 张牌。判断谁是赢家。

### 【输入格式】

三行, 分别是字符串  $S_A, S_B, S_C$ 。

### 【输出格式】

如果 A 胜出输出 ‘A’, 如果 B 胜出输出 ‘B’, 如果 C 胜出输出 ‘C’。

### 【输入样例#1】

```
aca
accc
ca
```

### 【输出样例#1】

```
A
```

### 【输入样例#2】

```
abcb
aacb
bccc
```

### 【输出样例#2】



C

**【数据说明】**

字符串  $S_A, S_B, S_C$  长度不超过 100，只包含 ‘a’，‘b’，‘c’ 三种字符。对于输出样例#1，游戏过程如下：

- A 出牌 ‘a’，之后轮到 A。
- A 出牌 ‘c’，之后轮到 C。
- C 出牌 ‘c’，之后轮到 C。
- C 出牌 ‘a’，之后轮到 A。
- A 出牌 ‘a’，之后轮到 A。
- 此时 A 手上已经没有牌了，A 是赢家。

## 例题 5.2 ABC084C Special Trains

### 【题目描述】

Atcoder 国新建成了一条铁路，铁路沿东西方向，沿线共有  $N$  站，从西到东依次编号为  $1, 2, \dots, N$

明天铁路将进行通车式，所有列车都是从西向东运行，并且只在相邻的站点之间有列车运行，从第  $i$  站到第  $i+1$  站的列车全程需要花费  $C_i$  秒。

第  $i$  站出发开往第  $i+1$  站的第一班列车在通车式开始后  $S_i$  秒发车，之后每过  $F_i$  秒发出一班列车（保证  $S_i$  一定是  $F_i$  的倍数）

如果不计换乘的时间，请计算从第  $i$  站出发的乘客到达最后的第  $N$  站的时刻，最早可能在通车式开始后多少秒。

### 【输入格式】

第一行一个正整数  $N$

接下来  $N-1$  行，每行三个正整数  $C_i, S_i, F_i$

### 【输出格式】

输出  $N$  行，第  $i$  行输出从第  $i$  站出发的乘客到达第  $N$  站的最早时间。

### 【输入样例#1】

```
3
6 5 1
1 10 2
```

### 【输出样例#1】

```
13
11
0
```

### 【输入样例#2】

```
4
12 24 6
52 16 4
99 2 2
```

### 【输出样例#2】

```
187
167
101
0
```

**【数据说明】**

$$1 \leq N \leq 500$$

$$1 \leq C_i \leq 100$$

$$1 \leq S_i \leq 10^5$$

$$1 \leq F_i \leq 100$$

保证  $S_i$  一定是  $F_i$  的倍数

样例 1:

从第 1 站出发的乘客旅行过程如下:

第 5 秒登上列车前往第 2 站, 第 11 秒到达, 但是从 2 到 3 的列车从第 10 秒开始每 2 秒发车一次, 需要等到第 12 秒登上列车前往 3 站, 第 13 秒到达

从第 2 站出发的乘客旅行过程如下:

第 10 秒登上列车前往 3 站, 第 11 秒到达

对于从第 3 站出发的乘客也要输出一个 0

### 例题 5.3 洛谷 P5016 龙虎斗

#### 【题目描述】

轩轩和凯凯正在玩一款叫《龙虎斗》的游戏，游戏的棋盘是一条线段，线段上有  $n$  个兵营（自左至右编号  $1 \sim n$ ），相邻编号的兵营之间相隔 1 厘米，即棋盘为长度为  $n-1$  厘米的线段。 $i$  号兵营里有  $c_i$  位工兵。下面图 1 为  $n=6$  的示例：



图 1.  $n=6$  的示例

轩轩在左侧，代表“龙”；凯凯在右侧，代表“虎”。他们以  $m$  号兵营作为分界，靠左的工兵属于龙势力，靠右的工兵属于虎势力，而第  $m$  号兵营中的工兵很纠结，他们不属于任何一方。

一个兵营的气势为：该兵营中的工兵数  $\times$  该兵营到  $m$  号兵营的距离；参与游戏一方的势力定义为：属于这一方所有兵营的气势之和。下面图 2 为  $n=6, m=4$  的示例，其中红色为龙方，黄色为虎方：

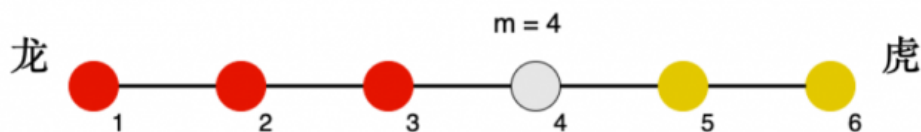


图 2.  $n=6, m=4$  的示例

游戏过程中，某一刻天降神兵，共有  $s_1$  位工兵突然出现在了  $p_1$  号兵营。作为轩轩和凯凯的朋友，你知道如果龙虎双方气势差距太悬殊，轩轩和凯凯就不愿意继续玩下去了。为了让游戏继续，你需要选择一个兵营  $p_2$ ，并将你手里的  $s_2$  位工兵全部派往兵营  $p_2$ ，使得双方气势差距尽可能小。

注意：你手中的工兵落在哪个兵营，就和该兵营中其他工兵有相同的势力归属（如果落在  $m$  号兵营，则不属于任何势力）。

**【输入格式】**

输入文件的第一行包含一个正整数  $n$ ，代表兵营的数量。

接下来的一行包含  $n$  个正整数，相邻两数之间以一个空格分隔，第  $i$  个正整数代表编号为  $i$  的兵营中起始时的工兵数量  $c_i$ 。

接下来的一行包含四个正整数，相邻两数间以一个空格分隔，分别代表  $m, p_1, s_1, s_2$ 。

**【输出格式】**

输出文件有一行，包含一个正整数，即  $p_2$ ，表示你选择的兵营编号。如果存在多个编号同时满足最优，取最小的编号。

**【输入样例#1】**

```
6
2 3 2 3 2 3
4 6 5 2
```

**【输出样例#1】**

```
2
```

**【输入样例#2】**

```
6
1 1 1 1 1 16
5 4 1 1
```

**【输出样例#2】**

```
1
```

### 【输入输出样例 1 说明】

见问题描述中的图 2。双方以  $m = 4$  号兵营分界，有  $s_1 = 5$  位工兵突然出现在  $p_1 = 6$  号兵营。

龙方的气势为：

$$2 \times (4 - 1) + 3 \times (4 - 2) + 2 \times (4 - 3) = 14$$

虎方的气势为：

$$2 \times (5 - 4) + (3 + 5) \times (6 - 4) = 18$$

当你将手中的  $s_2 = 2$  位工兵派往  $p_2 = 2$  号兵营时，龙方的气势变为：

$$14 + 2 \times (4 - 2) = 18$$

此时双方气势相等。

### 【输入输出样例 2 说明】

双方以  $m = 5$  号兵营分界，有  $s_1 = 1$  位工兵突然出现在  $p_1 = 4$  号兵营。

龙方的气势为：

$$1 \times (5 - 1) + 1 \times (5 - 2) + 1 \times (5 - 3) + (1 + 1) \times (5 - 4) = 11$$

虎方的气势为：

$$16 \times (6 - 5) = 16$$

当你将手中的  $s_2 = 1$  位工兵派往  $p_2 = 1$  号兵营时，龙方的气势变为：

$$11 + 1 \times (5 - 1) = 15$$

此时可以使双方气势的差距最小。

### 【数据规模与约定】

$1 < m < n, 1 \leq p_1 \leq n$ 。

对于 20% 的数据， $n = 3, m = 2, c_i = 1, s_1, s_2 \leq 100$ 。

另有 20% 的数据， $n \leq 10, p_1 = m, c_i = 1, s_1, s_2 \leq 100$ 。

对于 60% 的数据， $n \leq 100, c_i = 1, s_1, s_2 \leq 100$ 。

对于 80% 的数据， $n \leq 100, c_i, s_1, s_2 \leq 100$ 。

对于 100% 的数据， $n \leq 10^5, c_i, s_1, s_2 \leq 10^9$ 。

## 例题 5.4 P1532 卡布列克圆舞曲

## 【题目描述】

卡布列克是一位数学家，他在研究数字时发现：任意一个不是用完全相同数字组成的四位数，如果对它们的每位数字重新排序，组成一个较大的数和一个较小的数，然后用较大数减去较小数，差不够四位数时补零，类推下去，最后将变成一个固定的数：6174，这就是卡布列克常数。

例如：

$$4321 - 1234 = 3087$$

$$8730 - 378 = 8352$$

$$8532 - 2358 = 6174$$

$$7641 - 1467 = 6174$$

如果  $K$  位数也照此办理，它们不是变成一个数，而是在几个数字之间形成循环，称作卡布列克圆舞曲。例如对于五位数 54321：

$$54321 - 12345 = 41976$$

$$97641 - 14679 = 82962$$

$$98622 - 22689 = 75933$$

$$97533 - 33579 = 63954$$

$$96543 - 34569 = 61974$$

$$97641 - 14679 = 82962$$

我们把 82962, 75933, 63954, 61974 称作循环节，即卡布列克圆舞曲。

## 【输入格式】

输入包含  $N$  组测试数据

第1行为一个正整数  $N$ 。

接下来  $N$  行，每行为一个待求"卡布列克圆舞曲"的起始整数（小于  $10^{18}$ ）

## 【输出格式】

每行为对应整数的循环节，数据之间用空格隔开。

## 【输入样例#1】

```
2
4321
54321
```

## 【输出样例#1】

```
6174
82962 75933 63954 61974
```

## 【数据说明】

$$1 \leq N \leq 100$$

所有输入小于  $10^{18}$

## 第六章 阶段练习1



## 第七章 递归

### 7.1 函数递归

“递归”一般指的是把一个问题（或定义）转化为一个或多个更简单的同类问题（或定义），通过解决（或解释）这些更简单的情况来解决（或解释）原先的情况。

函数是可以在函数内部调用自身：在一个函数中再次调用该函数自身的行为叫做函数递归，这样的函数被称作递归函数。当问题可以递归定义的时候，通常也可以递归解决，而递归解决则经常是用递归函数实现的。

例如，阶乘可以进行如下的递归定义：

$$n! = \begin{cases} 1 & n = 0 \\ n \times (n-1)! & n > 0 \end{cases}$$

因此，对于“计算整数  $n$  的阶乘”的问题，当  $n$  等于 0 时，此问题足够简单可以直接求解；当  $n$  大于 0 时，求  $n!$  可以转化为求  $(n-1)!$  这个更简单的问题，这就形成了一个解决该问题的递归方法。

下面的递归函数实现了“计算整数  $n$  的阶乘”的问题。

```
1 int fact(int n)
2 {
3     if (n == 0) return 1;
4     return n * fact(n - 1);
5 }
```

当在主函数中调用  $f(3)$  时，函数第1次被调用时  $n = 3$ ，接下来函数不是直接计算函数值而是继续调用  $f(3 - 1)$ 。第2次调用时  $n = 2$ ，由于  $n$  不等于 0，此时函数仍然不计算函数值，继续调用  $f(2 - 1)$ 。第3次调用  $f(1)$  时  $n = 1$ ，由于  $n$  不等于 0，此时函数仍然不计算函数值，继续调用  $f(1 - 1)$ 。第4次调用时  $n = 0$ ，此时函数执行 `return 1` 语句，将函数值  $f(0) = 1$  返回给第3次调用。第3次调用计算  $1 \times f(1 - 1)$  的值，将函数值  $f(1) = 1$  返回给第2次调用。第2次调用计算  $2 \times f(2 - 1)$  的值将函数值  $f(2) = 2$  返回给第1次调用。第1次调用计算  $3 \times f(3 - 1)$  的值将函数值  $f(3) = 6$  返回到主函数。

需要注意，函数递归时，前面的函数调用在函数返回为止，一直处于“等待后面的函数调用返回值”的状态。例如上面的例子中，第2次调用必须等待第3次调用返回了函数值 1 之后，才能计算自己的函数值，并返回给上一次调用。

通常而言，对于一个问题,如果它存在如下特点，那么这个问题就可以用递归来求解：

- (1) 该问题存在一个或多个可以直接给出答案的“简单情况”。这一般称作递归的出口；
- (2) 对于其他情况，该问题可以被分解为更简单的同类问题。也就是说问题必须是可转化的，转化必须是单方向的；
- (3) 从任何一种情况出发，总会递归得到“简单情况”，这保证了递归会结束。

在写递归函数时，一般会用参数指出函数正在解决哪个情况，并用类似下面这样的选择语句，根据参数区分简单情况和需要递归的情况。

```
1 if ( 简单情况 ) {
2     直接处理
3 }
4 else {
5     递归
6 }
```

## 例题 7.1 完成下列任务

1. 写一个递归函数 `int gcd(int a, int b)`, 递归求  $a, b$  的最大公约数;
2. 写一个递归函数 `int fab(int k)`, 递归求斐波那契数列第  $k$  项;
3. 写一个递归函数 `int comb(int n, int m)`, 递归求组合数  $C_n^m$ ;

利用递推式  $C_n^m = C_{n-1}^{m-1} + C_{n-1}^m$  计算组合数  $C_n^m$

## 例题 7.2 CF727A 变形：从 A 到 B (简单版)

## 【题目描述】

瓦西里有一个数  $a$ ，他想把这个数变成  $b$ ，为此他可以做两种变换：

1. 把现有的数  $x$  变为  $2x$
2. 把现有的数  $x$  后面接一个 1 (即  $x$  变为  $10x + 1$ )

例如  $a = 2, b = 162$ ，你可以以如下的方式把  $a$  变成  $b$

$2 \rightarrow 4 \rightarrow 8 \rightarrow 81 \rightarrow 16$

你需要帮瓦西里判断一下，把  $a$  变成  $b$  是否可能。

## 【输入格式】

输入共 1 行：

第 1 行，两个整数  $a, b$ ，意义如题目描述。

## 【输出格式】

输出共 1 行：

第 1 行，一个字符串，如果变形是可能的，就输出 "YES"；否则输出 "NO" (均不带引号)

## 【输入样例#1】

2 162

## 【输出样例#1】

YES

## 【数据说明】

$1 \leq a, b \leq 10^9$

## 7.2 搜索

我们先来看下面一个暴力枚举的例子：

**例题 7.3** 你有5个重量不同的砝码和一个天平，每个砝码可以放在左盘或者右盘或者不放。

砝码重量为  $w_1, w_2, w_3, w_4, w_5$

问有多少种放置砝码的方式，使得天平平衡？

对于这个问题，我们可以枚举每个砝码放在哪里。为此我们写出 5 重循环：

```
1 //用数组a记录每个砝码放在哪里,0表示放左盘,2表示放右盘,1表示不放
2 for(a[1]=0;a[1]<=2;a[1]++)
3 for(a[2]=0;a[2]<=2;a[2]++)
4 for(a[3]=0;a[3]<=2;a[3]++)
5 for(a[4]=0;a[4]<=2;a[4]++)
6 for(a[5]=0;a[5]<=2;a[5]++){
7     //遍历数组a,计算左右盘砝码总重
8     int left=0,right=0;
9     for(int i=1;i<=5;i++){
10         if(a[i]==0) left+=w[i];
11         if(a[i]==2) right+=w[i];
12     }
13     if(left==right) cnt++;
14 }
```

这个程序有一个缺点，只能处理 5 个砝码的问题，如果改成 6 个砝码，我们就需要改动源代码，增加一层循环。那么有没有不需改动源代码，就能处理任意多砝码的方法呢？这就需要用到函数递归。

例题 7.4 你有  $n$  个重量不同的砝码和一个天平，每个砝码可以放在左盘或者右盘或者不放。

砝码重量为  $w_1, w_2, \dots, w_n$

问有多少种放置砝码的方式，使得天平平衡？

有  $n$  个砝码，我们可以把一层循环写成一个递归函数。在函数中枚举第  $p$  步的选择：将砝码放在哪里。如果还没有达到第  $n$  步，就调用自身，多嵌套一层循环；如果达到了  $n$  步，就计算左右砝码总重并判断。

代码如下：

```
1 //递归函数，参数 p 表示当前搜索到的步数
2 void search(int p){
3     if(p > n){//这说明前 n 层循环都嵌套好了，可以判断了
4         int left = 0, right = 0;
5         for(int i = 1; i <= n; i++){
6             if(a[i] == 0) left += w[i];
7             if(a[i] == 2) right += w[i];
8         }
9         if(left == right) cnt++;
10        return;//这里要返回，不要再执行下面的嵌套循环代码
11    }
12    //枚举当前第 p 步的所有做法
13    for(int i = 0; i <= 2; i++){
14        a[p] = i;
15        search(p + 1);//嵌套下一层循环
16    }
17 }
```

需要注意的是，上面的函数中，使用了变量 `n`、`cnt` 和数组 `a`、`w`，所以这四个的定义应该写在函数前面，完整代码如下：

```
1  #include<iostream>
2  #include<cstdio>
3  using namespace std;
4  const int MR=15;
5  int n, a[MR], w[MR], cnt;
6  //递归函数，参数 p 表示当前搜索到的步数
7  void search(int p){
8      if(p > n){//这说明前 n 层循环都嵌套好了,可以判断了
9          int left = 0, right = 0;
10         for(int i = 1; i <= n; i++){
11             if(a[i] == 0) left += w[i];
12             if(a[i] == 2) right += w[i];
13         }
14         if(left == right) cnt++;
15         return;//这里要返回,不要再执行下面的嵌套循环代码
16     }
17     //枚举当前第 p 步的所有做法
18     for(int i = 0; i <= 2; i++){
19         a[p] = i;
20         search(p + 1);//嵌套下一层循环
21     }
22 }
23 int main(){
24     cin>>n;
25     for(int i = 1; i <= n; i++) cin >> w[i];
26     cnt = 0;
27     search(1);//从第1层循环开始
28     cout << cnt << endl;
29     return 0;
30 }
```

### 例题 7.5 子集枚举

#### 【题目描述】

小明的班级有  $n$  名学生，编号为  $1 \sim n$ ，现在要选出一些人组队（数量不限）。例如有 4 个人，选出编号 1, 2, 4 这三个人组队，就是一种选择方法。我们将整个班的学生称作一个集合，选出来的队伍就叫做子集。也可以一个人也不选，这样队伍中一个人都没有，我们把这种子集称作空集。

我们把编号  $1 \sim n$  的学生看做整数  $1 \sim n$ 。写一个递归函数，枚举  $1 \sim n$  中取若干个数的每种方法（子集枚举）。对每种方法把取出的数从小到大排序，若两种方法中前  $k-1$  个数的选取情况相同，则取了第  $k$  个数的在前。例如对  $n=4$ ，应该按如下顺序给出结果：

{1, 2, 3, 4}, {1, 2, 3}, {1, 2, 4}, {1, 2}, {1, 3, 4}, {1, 3}, {1, 4}, {1}, {2, 3, 4}, {2, 3}, {2, 4}, {2}, {3, 4}, {3}, {4}, {}。

#### 【输入格式】

一个正整数  $n$

#### 【输出格式】

输出  $2^n$  行，每行一个子集，按题目要求顺序。

#### 【输入样例#1】

4

#### 【输出样例#1】

```
1 2 3 4
1 2 3
1 2 4
1 2
1 3 4
1 3
1 4
1
2 3 4
2 3
2 4
2
3 4
3
4
```

#### 【数据说明】

$1 \leq n \leq 12$ ；对于空集,需要输出一个空行



## 例题 7.6 货币面值

## 【题目描述】

小猴有  $n$  种面值不同的钱币，具体地说，小猴有面值  $w_1$  的钱币  $c_1$  个，面值  $w_2$  的钱币  $c_2$  个，……，面值  $w_n$  的钱币  $c_n$  个。

小猴想知道用手中的钱币，恰好凑出  $m$  的方法有多少种？

## 【输入格式】

第 1 行两个正整数  $n, m$ ，用空格分隔

第 2 行  $n$  个正整数  $w_1, w_2, \dots, w_n$

第 3 行  $n$  个正整数  $c_1, c_2, \dots, c_n$

## 【输出格式】

一个整数，恰好凑出  $m$  的方法数

## 【输入样例#1】

```
3 10
1 2 4
3 3 3
```

## 【输出样例#1】

```
4
```

## 【样例说明】

小猴有面值 1 的 3 个，面值 2 的 3 个，面值 4 的 3 个，恰好凑成 10 的方法有以下 4 种：

$4 + 4 + 2$

$4 + 4 + 1 + 1$

$4 + 2 + 2 + 2$

$4 + 2 + 2 + 1 + 1$

## 【数据说明】

$1 \leq n \leq 10$ ;

$1 \leq m, w_i \leq 10^6$ ,  $w_i$  互不相同;

$1 \leq c_i \leq 5$ ;

### 例题 7.7 子集枚举2

#### 【题目描述】

小明的班级有  $n$  名学生，他们的学号分别是  $a_1, a_2, \dots, a_n$ ，现在要选出一些人组队（数量不限）。

我们把学号  $a_1, a_2, \dots, a_n$  的学生看做整数  $a_1, a_2, \dots, a_n$ 。写一个递归函数。枚举  $n$  个数  $a_1, \dots, a_n$  中取若干个数的每种方法（子集枚举）。对每种方法把取出的数从小到大排序，若两种方法中前  $k-1$  个数的选取情况相同，则取了第  $k$  个数的在前。例如对  $n=4$ ， $a=\{100, 20, 3, 4\}$ ，应该按如下顺序给出结果

$\{100, 20, 3, 4\}$ ,  $\{100, 20, 3\}$ ,  $\{100, 20, 4\}$ ,  $\{100, 20\}$ ,  $\{100, 3, 4\}$ ,  $\{100, 3\}$ ,  
 $\{100, 4\}$ ,  $\{100\}$ ,  $\{20, 3, 4\}$ ,  $\{20, 3\}$ ,  $\{20, 4\}$ ,  $\{20\}$ ,  $\{3, 4\}$ ,  $\{3\}$ ,  $\{4\}$ ,  $\{\}$

#### 【输入格式】

第一行一个正整数  $n$

第二行  $n$  个正整数， $a_1, \dots, a_n$

#### 【输出格式】

输出  $2^n$  行，每行一个子集，按题目要求顺序

#### 【输入样例#1】

4  
 100 20 3 4

#### 【输出样例#1】

100 20 3 4  
 100 20 3  
 100 20 4  
 100 20  
 100 3 4  
 100 3  
 100 4  
 100  
 20 3 4  
 20 3  
 20 4  
 20  
 3 4  
 3  
 4

#### 【数据说明】

$1 \leq n \leq 12$ ;  $1 \leq a_i \leq 10^7$ ; 对于空集，需要输出一个空行

## 第八章 step型搜索

### 8.1 step型搜索

#### 例题 8.1 Bookshelf2

##### 【题目描述】

所有  $N(1 \leq N \leq 20)$  头奶牛都有一个确定的身高  $H_i(1 \leq H_i \leq 1,000,000)$ 。设所有奶牛身高的和为  $S$ 。书架的高度为  $B$ ，并且保证  $1 \leq B \leq S$ 。

为了够到比最高的那头奶牛还要高的书架顶，奶牛们不得不象演杂技一般，一头站在另一头的背上，叠成一座“奶牛塔”。当然，这个塔的高度，就是塔中所有奶牛的身高之和。为了往书架顶上放东西，所有奶牛的身高和必须不小于书架的高度。塔叠得越高便越不稳定，于是奶牛们希望找到一种方案，使得叠出的塔在高度不小于书架高度的情况下，高度尽可能小。

你也可以猜到你的任务了：写一个程序，计算奶牛们叠成的塔在满足要求的情况下，最少要比书架高多少。

##### 【输入格式】

第 1 行: 2 个用空格隔开的整数:  $N$  和  $B$

第 2 ~  $N + 1$  行: 第  $i + 1$  行是 1 个整数:  $H_i$

##### 【输出格式】

奶牛们叠成的塔最少比书架高的高度

##### 【输入样例#1】

5 16

3

1

3

5

6

##### 【输出样例#1】

1

##### 【样例#1说明】

我们选用奶牛 1、3、4、5 叠成塔，她们的总高度为  $3 + 3 + 5 + 6 = 17$ 。任何方案都无法叠出高度为 16 的塔，于是答案为 1。

## 例题 8.2 三角形牧场

## 【题目描述】

和所有人一样，奶牛喜欢变化。它们正在设想新造型的牧场。奶牛建筑师 Hei 想建造围有漂亮白色栅栏的三角形牧场。她拥有  $n$  块木板，每块的长度  $l_i$  都是整数，她想用所有的木板围成一个三角形使得牧场面积最大。

请帮助 Hei 小姐构造这样的牧场，并计算出这个最大牧场的面积。

## 【输入格式】

第 1 行：一个整数  $n$ ；

第 2 到第  $(n+1)$  行，每行一个整数，第  $(i+1)$  行的整数  $l_i$  表示第  $i$  块木板的长度。

## 【输出格式】

仅一个整数：最大牧场面积乘以 100 然后舍尾的结果。如果无法构建，输出  $-1$ 。

## 【输入样例#1】

```
5
1
1
3
3
4
```

## 【输出样例#1】

```
692
```

## 【数据说明】

对于 100% 的数据，保证  $3 \leq n \leq 40$ ， $1 \leq l_i \leq 40$ 。

## 例题 8.3 特殊子集检验

## 【题目描述】

数组  $a$  中的元素为  $\{a_1, \dots, a_n\}$ ，如果满足如下条件，就称这个数组是一个特殊和数组：

从数组  $a$  中取出若干数，取出的数称作  $a$  的一个子集，取出的数之和称作这个子集的和。

例如  $a = \{4, 5, 6, 7, 8\}$ ，从中取出  $\{4, 7, 8\}$  三个数作为一个子集，这个子集的和就是  $4 + 7 + 8 = 19$ 。

枚举从数组  $a$  中取出若干数的所有方法，如果这些方法中存在两种不同的取法，使得取出的数构成的子集和相等，那么这个数组就不是一个特殊和数组。

如果对于所有子集，这些子集的子集和都互不相同，这个数组就是一个特殊和数组。

例如， $\{4, 5, 6, 7, 8\}$  不是一个特殊和数组，因为  $4 + 5 + 6 = 7 + 8$ ； $\{14, 15, 16, 17, 18\}$  也不是一个特殊和数组，因为  $14 + 18 = 15 + 17$ ；而  $\{14, 15, 26, 27, 28\}$  的所有子集和都互不相同，是一个特殊和数组。

输入中包含了若干组3至12个元素的数组，对每一个数组，判断是否为特殊和数组。

## 【输入格式】

第1行，1个正整数  $T$ ，表示有  $T$  个数组

接下来  $T$  行，每行第1个数为  $n$ ，表示数组中元素的个数，后面  $n$  个数表示数组中的元素  $a_1, \dots, a_n$ 。

## 【输出格式】

对每个数组，判断是否为特殊和数组，是输出Yes，不是输出No

## 【输入样例#1】

```
3
5 4 5 6 7 8
5 14 15 16 17 18
5 14 15 26 27 28
```

## 【输出样例#1】

```
No
No
Yes
```

## 【数据说明】

$$1 \leq T \leq 20$$

$$2 \leq n \leq 12$$

$$1 \leq a_i \leq 10^8$$

## 8.2 剪枝

首先考虑这个问题:

### 例题 8.4 “123”好数

一个数中如果如果不含连续的“12”，连续的“23”，连续的“31”，就称作“123”好数。那么，在由 123 构成的  $n$  位数中，第  $k$  个“123”好数是什么？

我们可以通过搜索，在最深的一层递归函数中得到所有由 123 构成的  $n$  位数。然后去判断这个  $n$  位数是否含有“12”，“23”，“31”。例如 6 位数 122331 就含有不能出现的片段。

但是通过枚举我们发现，从 121111 到 123333 的所有数，因为开头都是“12”，都含有不能出现的片段“12”，所以实际上我们是不需要再枚举后 4 位的。此时我们应该直接进入下一次循环，去枚举以“13”开头的数。

像这样，在循环枚举的过程中，虽然还没有枚举出整体的状况，但是发现目前的一个部分已经不满足要求的话，可以跳过后面部分的枚举，通过 `continue` 直接进入下一次循环。这种操作叫做剪枝。

例题 8.5 “ $d$  好数” (计数)

## 【题目描述】

一个只由  $1 \sim m$  数码组成的  $n$  位数, 如果任意两个相邻数之差的绝对值都不超过  $d$ , 则称这个数是一个 “ $d$  好数”。

给出  $n, m, d$ , 输出  $n$  位 “ $d$  好数” 的个数。 $d$  的值只可能取 1, 2, 3

## 【输入格式】

1 行, 3 个正整数  $n, m, d$

## 【输出格式】

输出  $n$  位 “ $d$  好数” 的个数。

## 【输入样例#1】

5 3 1

## 【输出样例#1】

99

## 【数据说明】

$1 \leq n \leq 10; \quad 1 \leq m \leq 9; \quad 1 \leq d \leq 3。$

## 例题 8.6 倍数序列

## 【题目描述】

要求找到一个正整数序列，满足序列中所有的数不超过  $n$ ，序列长度为  $m$ ，且除了第一个数外，所有的数都能被前一个数整除（即是前一个数的倍数）。

我们想要知道这样的序列都有哪些，输出所有满足要求的序列。

## 【输入格式】

第一行，两个正整数  $n, m$ 。

## 【输出格式】

按字典序从小到大，每行输出一个满足要求的序列，数与数之间用空格分隔。

## 【输入样例#1】

4 3

## 【输出样例#1】

```
1 1 1
1 1 2
1 1 3
1 1 4
1 2 2
1 2 4
1 3 3
1 4 4
2 2 2
2 2 4
2 4 4
3 3 3
4 4 4
```

## 【样例说明】

以下 13 个数列满足条件

1, 1, 1 ; 1, 1, 2 ; 1, 1, 3 ; 1, 1, 4 ;  
1, 2, 2 ; 1, 2, 4 ; 1, 3, 3 ; 1, 4, 4 ;  
2, 2, 2 ; 2, 2, 4 ; 2, 4, 4 ;  
3, 3, 3 ; 4, 4, 4 。

## 【数据说明】

$1 \leq n \leq 40 ; 1 \leq m \leq 20$



例题 8.7 第  $k$  个上升数

## 【题目描述】

从左往右, 如果每一位数字都大于等于其左边的数字, 这样的数被称为上升数, 比如 134468。  
给出  $n$  和  $k$ , 输出所有  $n$  位的上升数中从小到大排列后的第  $k$  个数。

## 【输入格式】

2 个正整数  $n, k$

## 【输出格式】

输出从小到大第  $k$  个  $n$  位上升数。

## 【输入样例#1】

3 30

## 【输出样例#1】

149

## 【输入样例#2】

28 152156

## 【输出样例#2】

11111111111111112234568888999

## 【数据说明】

$$3 \leq n \leq 30$$

$$1 \leq k \leq 10^7$$

保证  $k$  不超过  $n$  位上升数的个数

## 第九章 加参数剪枝

除了用一个数组记录搜索过程中每一步的选择，我们还可以使用一些变量来记录搜索过程中的其他值。例如整数拆分问题：选出若干个数使得总和等于  $n$ 。我们可以用一个变量 `sum` 记录当前选出的所有数之和，并且将 `sum` 作为参数。这样我们就可以在 `sum` 大于  $n$  时，及时剪枝。

### 例题 9.1 整数划分问题I

#### 【题目描述】

你将得到一个整数  $n$ ，请列出把整数  $n$  划分为  $k$  个正整数的每一种方法。

#### 【输入格式】

输入共 1 行

第 1 行，2 个正整数  $n, k$

#### 【输出格式】

输出为若干行：

每行为用空格隔开的若干个正整数，为一种  $n$  划分为  $k$  个数的方法，每组数按从小到大输出。

若两种方法中前  $k-1$  个数相同，则第  $k$  个数更小的在前。

#### 【输入样例】

```
1 6 3
```

#### 【输出样例】

```
1 1 1 4
```

```
2 1 2 3
```

```
3 2 2 2
```

#### 【数据说明】

$1 \leq k \leq n \leq 20$

## 例题 9.2 整数划分问题II

## 【题目描述】

你将得到一个整数  $n$ ，请列出把整数  $n$  划分为若干个正整数的每一种方法。

## 【输入格式】

输入共 1 行；

第 1 行，1 个正整数  $n$ ；

## 【输出格式】

输出为若干行：

每行为用空格隔开的若干个正整数，为一种  $n$  划分为若干个正整数的方法，每组数按从小到大输出。

若两种方法中前  $k-1$  个数相同，则第  $k$  个数更小的在前。

## 【输入样例】

```
1 5
```

## 【输出样例】

```
1 1 1 1 1 1
2 1 1 1 2
3 1 1 3
4 1 2 2
5 1 4
6 2 3
7 5
```

## 【数据说明】

$1 \leq n \leq 20$

### 例题 9.3 整数划分问题III

#### 【题目描述】

你将得到一个整数  $n$ ，请列出把整数  $n$  划分为若干个正整数的每一种方法，这些正整数必须选自数组  $a_1, a_2, \dots, a_k$ 。

数组  $a$  中可能有重复元素，但是每个最多只可以选择一次，例如对  $n = 6, a = \{1, 1, 2, 2, 3, 4\}$ ， $n = 1 + 1 + 2 + 2$  是可以的，因为数组  $a$  中有2个1和2个2可供选择； $n = 2 + 2 + 2$  则不正确，因为数组  $a$  中没有3个2。

#### 【输入格式】

输入共2行

第1行，2个正整数  $n, k$ ； $k$  为数组  $a$  的元素个数；

第2行， $k$  个用空格隔开的正整数  $a_1, a_2, \dots, a_k$ ，为数组  $a$  中的元素；

#### 【输出格式】

输出为若干行：

每行为用空格隔开的若干个正整数，为一种  $n$  划分为若干个正整数的方法，每组数按在  $a$  中的下标从小到大输出。

若两种方法中前  $k - 1$  个数在  $a$  中的下标相同，则第  $k$  个数的下标更小的在前。

如果两种方法选取的数在  $a$  中的下标不全相同，则视为不同的方法。例如对  $n = 6$ ， $a = \{1, 1, 2, 2, 3, 4\}$ ，选取  $a$  的下标为 1, 3, 5 或 1, 4, 5 号元素都会得到  $n = 1 + 2 + 3$ ，两者都要在合适的位置分别输出。

#### 【输入样例】

```
1 5 15
2 5 5 10 2 3
```

#### 【输出样例】

```
1 5 5 2 3
2 5 10
3 5 10
4 10 2 3
```

#### 【数据说明】

$1 \leq n, a_1, a_2, \dots, a_k \leq 10^9$ ； $1 \leq k \leq 20$ 。

## 例题 9.4 CF727A 变形：从 A 到 B

## 【题目描述】

瓦西里有一个数  $a$ ，他想把这个数变成  $b$ ，为此他可以做两种变换：

1. 把现有的数  $x$  变为  $2x$
2. 把现有的数  $x$  后面接一个 1 (即  $x$  变为  $10x + 1$ )

例如  $a = 2, b = 162$ ，你可以以如下的方式把  $a$  变成  $b$

$2 \rightarrow 4 \rightarrow 8 \rightarrow 81 \rightarrow 162$

你需要帮瓦西里判断一下，把  $a$  变成  $b$  是否可能。

## 【输入格式】

输入共 1 行：

第 1 行，两个整数  $a, b$ ，意义如题目描述。

## 【输出格式】

输出共 3 行：

第 1 行，一个字符串，如果变形是可能的，就输出 "YES"；否则输出 "NO"(均不带引号)

第 2 行，一个整数  $k$ ，意义与第 3 行一起解释。

第 3 行，用空格隔开的  $k$  个正整数  $a_1, a_2, \dots, a_k$ ，表示把  $a$  变成  $b$  的过程，其中  $a = a_1, b = a_k$ ，过程中  $a_i$  会变成  $a_{i+1}$ 。

如果有多种可能的变化序列，认为越早使用变化 1 的越优先。请输出最优先的一组解。

## 【输入样例#1】

2 162

## 【输出样例#1】

YES

5

2 4 8 81 162

## 【数据说明】

$1 \leq a, b \leq 10^9$

## 例题 9.5 ABC114C 753数

**【题目描述】**

我们定义只由 357 三种数码组成，且包含所有三种数码的正整数为 753 数。

给出正整数  $N$ ，输出不超过  $N$  的 753 数的个数。

**【输入格式】**

一个正整数  $N$

**【输出格式】**

不超过  $N$  的 753 数的个数

**【输入样例#1】**

575

**【输出样例#1】**

4

**【输入样例#2】**

3600

**【输出样例#2】**

13

**【数据说明】**

$1 \leq N \leq 10^9$

## 例题 9.6 USACO-training1.5 超质数肋骨

## 【题目描述】

农民约翰的母牛总是产生最好的肋骨。你能通过农民约翰和美国农业部标记在每根肋骨上的数字认出它们。

农民约翰确定他卖给买方的是真正的质数肋骨，是因为从右边开始切下肋骨。每次还剩下的肋骨上的数字都组成一个质数。

举例来说：对于从左到右的四根肋骨 7,3,3,1，全部肋骨上的数字 7331 是质数；三根肋骨 733 是质数；二根肋骨 73 是质数；当然，最后一根肋骨 7 也是质数。因此 7331 被叫做长度 4 的特殊质数。

写一个程序对给定的肋骨的数目  $n$ ，求出所有的特殊质数。

## 【输入格式】

输入共 1 行：

第 1 行一个正整数  $n$ 。

## 【输出格式】

输出为若干行：

按从小到大的顺序输出长度为  $n$  的特殊质数，每行一个

## 【输入样例#1】

4

## 【输出样例#1】

2333

2339

2393

2399

2939

3119

3137

3733

3739

3793

3797

5939

7193

7331

7333

7393

## 【数据说明】

对于 100% 的数据： $1 \leq n \leq 8$ 。

## 例题 9.7 CF910A 回家的路

## 【题目描述】

直线上依次有  $1 \sim n$  号位置，相邻位置距离为 1，部分位置上有百合花，只有这些位置青蛙可以站上去。一只青蛙在 1 号位置，而它的家在  $n$  号位置，他每次可以跳  $1 \sim d$  的距离。你要计算青蛙至少跳几次可以到家。

## 【输入格式】

输入共 2 行：

第 1 行，两个用空格隔开的整数  $n, d$ ，意义如题目描述

第 2 行， $n$  长度的由 0,1 组成的字符串，依次表示  $1 \sim n$  号位置是否有百合花，1 表示有，0 表示无。

## 【输出格式】

输出共 1 行：

第 1 行，1 个整数，为青蛙到家至少需要跳的次数。如果它不可能回家则输出 -1。

## 【输入样例#1】

```
12 3
1011111100101
```

## 【输出样例#1】

```
4
```

## 【数据说明】

$$2 \leq n \leq 100; 1 \leq d \leq n - 1$$



## 第十章 回溯法

### 10.1 全排列枚举

#### 例题 10.1 全排列枚举

写一个递归函数，枚举  $1 \sim n$  的所有排列方式（全排列枚举）。若两种排列方法中前  $k-1$  个数相同，则第  $k$  个数更小的在前。例如对  $n=3$ ，应该按如下顺序给出结果

{1,2,3},{1,3,2},{2,1,3},{2,3,1},{3,1,2},{3,2,1}

我们用数组  $a$  来存放排列。在  $a[1]$  到  $a[p-1]$  已经填好的前提下，我们尝试把下一个数填入  $a[p]$ 。此时发生一个问题：已经填入  $a[1]$  到  $a[p-1]$  的数不能再用了，我们如何知道哪些数能用哪些不能用？

一个好的办法是使用  $\text{bool}$  数组。我们用  $\text{bool}$  数组  $v$  来标记每个数是否已经用过。接下来就可以从  $1$  到  $n$  枚举  $a[p]$  的值，如果遇到用过的数就 `continue`，如果遇到没用过的数，就令  $v[a[p]]=\text{true}$  并进入下一层循环。在下一层循环完全结束后，此时  $a[p]$  又恢复未使用状态，需要把  $v[a[p]]$  的值改回 `false`。代码如下：

```
1 #include<iostream>
2 using namespace std;
3 const int MR=15;
4 int n,a[MR];
5 bool v[MR]={};
6 //递归函数，参数p表示当前搜索的步数（将要枚举第p个数取什么）
7 void dfs(int p){
8     if(p==n+1){
9         for(int i=1;i<n;i++){
10             cout<<a[i]<<' ';
11         }
12         cout<<a[n]<<endl;
13         return;//这里要返回，不要再执行下面的嵌套循环代码
14     }
15     //枚举第p个数选择什么
16     for(int i=1;i<=n;i++){
17         //尝试在第p个数的位置填i
18         if(v[i]) continue;//如果这个数已经被用过了,就尝试下一个数
19         a[p]=i;
20         v[i]=true;//记录i这个值已经被用过了
```

```

21     search(p+1); //进行下一步搜索
22     v[i]=false; //一定要改回来!
23 }
24 }
25 int main(){
26     cin>>n;
27     dfs(1); //搜索从第1步开始
28     return 0;
29 }

```

## 例题 10.2 全排列枚举2

### 【题目描述】

写一个递归函数，枚举  $n$  个数  $a_1, \dots, a_n$  的所有排列方式（全排列枚举）。若两种排列方法中前  $k-1$  个数相同，则第  $k$  个数更小的在前。例如对  $n=3$ ,  $a=\{100, 20, 3\}$ ，应该按如下顺序给出结果

$\{100, 20, 3\}$ ,  $\{100, 3, 20\}$ ,  $\{20, 100, 3\}$ ,  $\{20, 3, 100\}$ ,  $\{3, 100, 20\}$ ,  $\{3, 20, 100\}$

### 【输入格式】

第一行一个正整数  $n$

第二行  $n$  个正整数,  $a_1, \dots, a_n$

### 【输出格式】

$n!$  行，每行一个排列，按题目要求顺序。

### 【输入样例#1】

```

3
100 20 3

```

### 【输出样例#1】

```

100 20 3
100 3 20
20 100 3
20 3 100
3 100 20
3 20 100

```

### 【数据说明】

$1 \leq n \leq 8$ ;  $1 \leq a_i \leq 10^7$ ;

## 例题 10.3 选书

## 【题目描述】

学校放寒假时，信息学奥赛辅导老师有  $1, 2, 3, \dots, x$  本书，要分给参加培训的  $x$  个人，每人只能选一本书，但是每人有两本喜欢的书。老师事先让每个人将自己喜欢的书填写在一张表上。然后根据他们填写的表来分配书本，希望设计一个程序帮助老师求出所有可能的分配方案，使每个学生都满意。

## 【输入格式】

第 1 行：一个数  $x$

第 2 行到第  $x+1$  行：每行两个数，表示第  $i$  个人喜欢的书的序号

## 【输出格式】

只有一个数：总方案数。

## 【输入样例#1】

```
5
1 3
4 5
2 5
1 4
3 5
```

## 【输出样例#1】

```
2
```

## 【数据说明】

所有数据： $x \leq 20$

## 例题 10.4 吃奶酪

## 【题目描述】

房间里放着 $n$ 块奶酪。一只小老鼠要把它们都吃掉，问至少要跑多少距离？老鼠一开始在 $(0,0)$ 点处。

## 【输入格式】

第一行一个数 $n(n \leq 15)$

接下来每行2个实数，表示第 $i$ 块奶酪的坐标。

两点之间的距离公式  $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

## 【输出格式】

一个数，表示要跑的最少距离，保留2位小数。

## 【输入样例#1】

```
4
1 1
1 -1
-1 1
-1 -1
```

## 【输出样例#1】

```
7.41
```

## 例题 10.5 素数环

## 【题目描述】

输入正整数  $n$ ，把整数  $1 \sim n$  组成一个环，使得相邻两个数之和均为质数。

输出所有可行的排列方案，输出时从整数 1 开始逆时针排列，按字典序从小到大输出。

## 【输入格式】

一个正整数  $n$

## 【输出格式】

每行输出一个可行的排列。

输出时从整数 1 开始逆时针排列，按字典序从小到大输出。

## 【输入样例#1】

6

## 【输出样例#1】

1 4 3 2 5 6

1 6 5 2 3 4

## 【数据说明】

$1 \leq n \leq 16$

## 第十一章 回溯搜索练习

### 例题 11.1 ABC119C 合成竹子

#### 【题目描述】

你现在有  $n$  根竹子，它们的长度分别是  $l_1, l_2, \dots, l_n$ （单位 cm）

你的目的是使用这些竹子制作出长度为  $A, B, C$  的三根竹子。你可以使用以下 3 种魔法：

- (1) 延长魔法：消耗 1MP，选择 1 根竹子，将它的长度增加 1cm
- (2) 缩短魔法：消耗 1MP，选择 1 根长度大于 1cm 的竹子，将它的长度减少 1cm
- (3) 合成魔法：消费 10MP，将两根竹子合成一根，长度等于两根竹子之和。

为了得到三根长度为  $A, B, C$  的竹子，最少需要消耗多少 MP？

#### 【输入格式】

第 1 行，4 个正整数  $n, A, B, C$

接下来  $n$  行，每行 1 个正整数  $l_i$

#### 【输出格式】

需要消耗的最少 MP

#### 【输入样例#1】

```
5 100 90 80
98
40
30
21
80
```

#### 【输出样例#1】

```
23
```

#### 【样例#1说明】

目标是得到长度 100, 90, 80 的竹子，长度为 80 的竹子一开始就有了。按如下顺序使用魔法可以得到长度 100, 90 的竹子

对长度 98 的竹子使用 2 次延长魔法，得到长度 100 的竹子。

对长度 40, 30 的竹子使用合成魔法，得到长度 70 的竹子。

对长度 21 的竹子使用 1 次缩短魔法，得到长度 20 的竹子。

对长度 70 和 20 的竹子使用合成魔法，得到长度 90 的竹子。

**【输入样例#2】**

```
4 100 90 80
100
100
90
80
```

**【输出样例#2】**

```
0
```

**【样例#2说明】**

需要的竹子一开始就都有了，不需要使用任何魔法，也不一定需要使用所有竹子。

**【输入样例#3】**

```
8 1000 800 100
300
333
400
444
500
555
600
666
```

**【输出样例#3】**

```
243
```

**【数据说明】**

$$3 \leq N \leq 8$$

$$1 \leq C < B < A \leq 1000$$

$$1 \leq l_i \leq 1000$$

## 例题 11.2 小猴玩木棒

## 【题目描述】

现给出一些木棒长度，小猴能否用给出的木棒(必须全部用完)组成一个正方形呢？

## 【输入格式】

第一行是一个整数  $n$  表示测试的组数，接下来  $n$  行表示每组测试数据。

每组测试数据的第一个数为  $m(4 \leq m \leq 20)$ ，接下来  $m$  个数  $a_i(1 \leq a_i \leq 1000)$  表示木棒的长度。

## 【输出格式】

对于每组测试数据，如果可以组成正方形输出“yes”，否则输出“no”。

## 【输入样例#1】

```
3
4 1 1 1 1
5 10 20 30 40 50
8 1 7 2 6 4 4 3 5
```

## 【输出样例#1】

```
yes
no
yes
```



## 例题 11.3 USACO-training 健康的荷斯坦

## 【题目描述】

农民约翰以拥有世界上最健康的奶牛为傲。他知道每种饲料中所包含的牛所需的最低的维他命量是多少。请你帮助农夫喂养他的牛，以保持它们的健康，使喂给牛的饲料的种数最少。

给出牛所需的最低的维他命量，输出喂给牛需要哪些种类的饲料，且所需的饲料剂量最少。

维他命量以整数表示，每种饲料最多只能对牛使用一次，数据保证存在解。

## 【输入格式】

第 1 行：一个整数  $V(1 \leq V \leq 25)$ ，表示需要的维他命的种类数。

第 2 行： $V$  个整数 ( $1 \leq \text{每个数} \leq 1000$ )，表示牛每天需要的每种维他命的最小量。

第 3 行：一个整数  $G(1 \leq G \leq 15)$ ，表示可用来喂牛的饲料的种数。

下面  $G$  行，第  $n$  行表示编号为  $n$  饲料包含的各种维他命的量的多少。

## 【输出格式】

输出只有一行，包括

牛必需的最小的饲料种数  $P$

后面有  $P$  个数，表示所选择的饲料编号（按从小到大排列）

如果有多个解，输出饲料序号最小的（即字典序最小）

## 【输入样例#1】

```
4
100 200 300 400
3
50 50 50 50
200 300 200 300
900 150 389 399
```

## 【输出样例#1】

```
2 1 3
```

#### 例题 11.4 驾车旅行

##### 【题目描述】

如今许多普通百姓家有了私家车，一些人喜爱自己驾车从一个城市到另一个城市旅游。自己驾车旅游时总会碰到加油和吃饭的问题，在出发之前，驾车人总要想方设法得到从一个城市到另一个城市路线上的加油站的列表，列表中包括了所有加油站的位置及其每升的油价（如 3.25 元/L）。驾车者一般都有以下的习惯：

每当驾驶员到达一个加油站时：

1. 如果油箱里的汽油已经不够达到下一个加油站或目的地，驾驶员必须在当前加油站停下来加油。
2. 如果汽油足够到达下一个加油站，但是油箱的汽油小于最大容量一半时，驾驶员可以选择加油或者不加油。
3. 如果汽油足够到达下一个加油站，且油箱的汽油大于等于最大容量一半时，驾驶员一定不会加油。

每次加油一定会将邮箱加满。驾驶员在等待加油的同时要花 20 元买吃的。加油站付钱总是精确到 0.1 元（四舍五入）。

现在要你帮忙做的就是编写一个程序，计算出驾车从一个城市到另一个城市的旅游在加油和吃饭方面最少的费用。

##### 【输入格式】

第一行是一个实数  $L$ ，是从出发地到目的地的距离（单位：公里）。

第二行是三个实数  $v, m, c$  和一个整数  $n$ ，其中第一个实数  $v$  是汽车油箱的最大容量（单位：升）；第二个实数  $m$  是汽车每升油能行驶的公里数；第三个实数是  $c$  汽车在出发地加满油箱时花掉的费用（单位：元）；一个整数  $n$  是 1 到 50 间的数，表示从出发地到目的地线路上加油站的数目。

接下来  $n$  行都是两个实数  $d_i, p_i$ ，第一个数表示从出发地到某一个加油站的距离（单位：公里）；第二个实数表示该加油站汽油的价格（单位：元）。

数据保证驾驶员一定可以到达目的地。一条线路上的加油站根据其到出发地的距离递增排列，并且都不会大于从出发地到目的地的距离。

##### 【输出格式】

输出一个实数，即精确到 0.1 元的最小的加油和吃饭费用。

##### 【输入样例#1】

```
600
40 8.5 128 3
200 3.52
350 3.45
500 365
```

##### 【输出样例#1】

```
13133.2
```

## 例题 11.5 ABC147C HonestOrUnkind2

## 【题目描述】

$N$  个人以  $1 \sim N$  编号。分为一定会说真话的“诚实的人”，和有可能说谎的“不诚实的人”两种。

第  $i$  个人做出了  $A_i$  条证言，第  $i$  个人做出的第  $j$  条证言用两个整数  $x_{i,j}, y_{i,j}$  表示。 $y_{i,j} = 1$  代表证言是“第  $x_{i,j}$  个人是诚实的人”， $y_{i,j} = 0$  代表证言是“第  $x_{i,j}$  个人是不诚实的人”。

$N$  个人中，“诚实的人”最多有多少个？

## 【输入格式】

第 1 行，1 个正整数  $N$

接下来  $N$  组数据，第  $i$  组数据代表第  $i$  个人的证言

每组数据的第 1 行是 1 个整数  $A_i$

接下来  $A_i$  行，每行两个整数  $x_{i,j}, y_{i,j}$

## 【输出格式】

“诚实的人”的最多人数

## 【输入样例#1】

```
3
1
2 1
1
1 1
1
2 0
```

## 【输出样例#1】

```
2
```

## 【样例#1说明】

一共有 3 个人。第 1 人的证言是“第 2 个人是诚实的人”；第 2 人的证言是“第 1 个人是诚实的人”；第 3 人的证言是“第 2 个人是不诚实的人”。

假设第 1, 2 个人是诚实的人，第 3 个人是不诚实的，不会和任何证言矛盾。最多可能有 2 个诚实的人。

## 【输入样例#2】

```
3
2
2 1
3 0
2
3 1
```

1 0

2

1 1

2 0

**【输出样例#2】**

0

**【样例#2说明】**

不论假设任何人是诚实的，都会产生矛盾。

**【数据说明】**

$$1 \leq N \leq 15$$

$$0 \leq A_i \leq N - 1$$

$$1 \leq x_{i,j} \leq N; x_{i,j} \neq i; x_{i,j_1} \neq x_{i,j_2} (j_1 \neq j_2)$$

$$y_{i,j} = 0, 1$$

## 第十二章 阶段练习2